

✓ IMPORTANT

1. Interrupt all existing kernels (otherwise you will get memory problems). To do so go to the navigation bar and click Kernel > Shut Down All Kernels
2. **After** step 1, start this notebook with the **Torch** Kernel. To do so go to the navigation bar and click Kernel > Change Kernel... > Select Torch > Click Select (Alternatively via the kernel button on the top right of the notebook).

```
import importlib

packages = [
    .... "transformers",
    .... "datasets",
    .... "sentencepiece",
    .... "accelerate",
    .... "evaluate",
    .... "rouge_score",
    .... "bert_score",
    .... "sentence_transformers",
    .... "spacy",
]

for pkg in packages:
    .... try:
    ....     importlib.import_module(pkg)
    ....     print(f"✅ {pkg} is installed")
    .... except ImportError:
    ....     print(f"❌ {pkg} is NOT installed")
```

```
✅ transformers is installed
✅ datasets is installed
✅ sentencepiece is installed
✅ accelerate is installed
❌ evaluate is NOT installed
❌ rouge_score is NOT installed
❌ bert_score is NOT installed
✅ sentence_transformers is installed
✅ spacy is installed
```

```
!pip install -U evaluate rouge_score bert_score
# transformers datasets sentencepiece accelerate sentence-transformers
```

```
Collecting evaluate
  Downloading evaluate-0.4.6-py3-none-any.whl.metadata (9.5 kB)
Collecting rouge_score
  Downloading rouge_score-0.1.2.tar.gz (17 kB)
  Preparing metadata (setup.py) ... done
Collecting bert_score
  Downloading bert_score-0.3.13-py3-none-any.whl.metadata (15 kB)
```

```
Requirement already satisfied: datasets>=2.0.0 in /usr/local/lib/python3.
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/d
Requirement already satisfied: dill in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3
Requirement already satisfied: tqdm>=4.62.1 in /usr/local/lib/python3.12/
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: multiprocessing in /usr/local/lib/python3.12/
Requirement already satisfied: fsspec>=2021.05.0 in /usr/local/lib/python
Requirement already satisfied: huggingface-hub>=0.7.0 in /usr/local/lib/p
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dis
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: six>=1.14.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: torch>=1.0.0 in /usr/local/lib/python3.12/
Requirement already satisfied: transformers>=3.0.0 in /usr/local/lib/pyth
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/di
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/d
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in /usr/local/l
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/l
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/pyt
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/p
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.1
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/pytho
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/di
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/lo
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/lo
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/l
Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in /usr/loca
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/loc
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.4.2 in /usr/loc
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/loca
Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/li
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/loc
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/
Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/
```

✓ The *Newsreader Chatbot*: news retrieval,
summarization and question answering

In this project, you will build a simple chat-based interface for news consumption, the *Newsreader Chatbot*. This chatbot will feature three modules under the hood:

1. a news retrieval system (the corresponding task is generally referred to as **document retrieval**),
2. an automatic summarizer (this task is called **summarization** or **summary generation**),
3. a question-answering system (the task is simply referred to as **question answering** or, in short, **QA**).

In the end, you (the user) will be able to query the *Newsreader Chatbot* with a short sentence indicating the topic you would like to read about. The chatbot will retrieve the most relevant news stories and it will summarise them. You will then be able to select one of the stories and ask questions about it.

The main techniques this project will cover are:

- *Semantic search*, an approach to information retrieval that relies on text embeddings. You will implement two methods, one based on sentence embeddings and the other based on document embeddings.
- *Conditional text generation*, the task of generating a piece of text—for example, a summary or an open-ended answer—given another conditioning piece of text—such as a news story, optionally paired with a question. This task is more generally referred to as Natural Language Generation (NLG) and can serve many use cases, from translation and paraphrasing to storytelling and dialogue response generation. As NLG systems, you will use both general-purpose pre-trained language models and language models specifically fine-tuned on summarization or question answering. You will compare different models using standard NLG evaluation metrics.

```
## Generic libraries

# Dataframes
import pandas as pd

# Scientific computing
import numpy as np

# PyTorch, a machine learning framework
import torch

# Basic Python tools
import itertools
from collections import defaultdict

# Pretty printing
import textwrap
```

```
# A smart progress meter
from tqdm.notebook import trange, tqdm

## Project-specific libraries

# Evaluation metrics (for document retrieval)
from sklearn.metrics import accuracy_score, top_k_accuracy_score

# Spacy, a multi-use NLP library -- here, for sentence segmentation
from spacy.lang.en import English

# A library for sentence embeddings
from sentence_transformers import SentenceTransformer, util

## HuggingFace libraries

# A library for easy-to-use Transformer language models (and more)
from transformers import pipeline, AutoModelForCausalLM, AutoTokenizer,

# A library for ready-to-use NLP evaluation metrics
#import evaluate

# A library to easily import datasets from the HuggingFace hub
from datasets import load_dataset
```

```
# Convenience function for pretty printing
# Please execute this cell.
def pprint(string, width=120):
    print(textwrap.fill(string, width=width))
```

```
# pprint demo
string = "We have compiled a dataset of over 10,000 CNN news articles f
print(string)
print()
pprint(string)
```

We have compiled a dataset of over 10,000 CNN news articles for the purp

We have compiled a dataset of over 10,000 CNN news articles for the purp
Hugging Face dataset hub for extra ease of use: [news-qa-summarization](#). E
humans with a summary as well as a list of questions and corresponding a
development and evaluation of the three submodules of the Newsreader Cha

✓ News data: preliminary analysis

We have compiled a dataset of over 10,000 CNN news articles for the purpose of this project. It is available through the Hugging Face dataset hub for extra ease of use: [news-qa-summarization](#). Each article in the dataset is annotated by humans with a summary as well as a list of questions and corresponding answers. We are going to use

this dataset for the development and evaluation of the three submodules of the *Newsreader Chatbot*.

Let's load the dataset.

```
dataset = load_dataset("glnmario/news-qa-summarization")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:9
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settin
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to acc
warnings.warn(

README.md: 100%                               715/715 [00:00<00:00, 14.6kB/
s]

data.jsonl: 100%                               40.7M/40.7M [00:02<00:00, 17.9MB/
s]
```

The dataset contains a single partition, which is called `train` by default, and consists of four features:

- `story`: the news story
- `questions`: a list of questions regarding the news story
- `answers`: the list of answers to `questions`
- `summary`: a brief summary of the news story

```
dataset
```

```
DatasetDict({
  train: Dataset({
    features: ['story', 'questions', 'answers', 'summary'],
    num_rows: 10388
  })
})
```

Let's take a look at one data point.

```
# The news story
pprint(dataset['train'][1]['story'])
```

```
(AOL Autos)  -- At the 2009 Detroit Auto Show, Chrysler, Mercedes-Benz,
vehicles of the future will look like. And the future of driving looks f
mph in around 4 seconds. Those in the know realize that sometime in the
trucks in the US will feature electric final drive systems. The motors us
batteries, fuel cells, on-board generators, and perhaps even the sun. Bu
inevitability of this reality. Given our current economic times, reality
```

ideas of what electric vehicles (or "EVs" for short) might actually look like. The electric vehicles Chrysler showed in Detroit, Michigan, were shown at the Detroit Auto Show. Each of these vehicles is a running prototype, not some pie-in-the-sky-wish. A group of engineers at Chrysler that develops the company's EVs. To date, there are three powered models, each quite different from the other: a Dodge Circuit EV, a Dodge Country minivan (front-wheel-drive), a Jeep Wrangler Unlimited (all-wheel-drive). Chrysler promises to offer at least one of these models in 2010. The Circuit EV photos show Chrysler approaches electric vehicles with simple plug-in vehicles that use similar electric drive motors (only varying in power output and battery management controller). Each plugs in to 110- or 220-volt household outlets and uses an on-board range-extending battery charger (a generator). This generator's initial battery charge has been spent (usually within a range of 40 miles). The plug-in vehicles have an estimated range of approximately 400 miles. The generator runs with exceptional efficiency. This technology is similar in concept to the Chevrolet Volt, a vehicle that should be ready for production in 2010. A Circuit EV carries a larger battery pack and no generator, so its range is 120 miles. Its large battery pack combined with compact dimensions and the electric motor can blast the car from zero-to-sixty mph in around four seconds, exceptional for a mass-production type. Mercedes-Benz used the 2009 Detroit Auto Show to show its approach was to develop one efficient body style, and then equip it with different packages. AOL Autos: Mercedes Stirling Moss photos Much of the hardware for the propulsion units is built into what Mercedes calls "sandwich-floor" architecture for mass-production cars. The design helps keep heavy components mounted low on the chassis and maximized interior room. All three Concept BlueZERO vehicles include a large battery pack that is said to deliver a range of 120 miles. The F-Cell Plus supplements the vehicle's range with a hydrogen fuel cell. The fuel cell pack that extends cruising range to 240 miles. The E-Cell Plus, with a 1.6-liter turbo diesel engine as a range extender. The key is the on-board generator powered by tiny 1-liter turbo diesel engines.

```
# The list of questions
dataset['train'][1]['questions']
```

```
['What debuts at the 2009 Detroit Auto Show?',
 'Who shows electric cars?',
 'Where is the Detroit Auto Show located?']
```

```
# The corresponding answers
dataset['train'][1]['answers']
```

```
[['electric vehicles'],
 ['Chrysler, Mercedes-Benz, Toyota and MINI'],
 ['Michigan, ']]
```

Exercise: Write code to print out the summary.

```
# The summary
dataset['train'][1]['summary']
```

```
'Latest technologies debut at the 2009 Detroit Auto Show .\nChrysler, Mercedes-Benz, Toyota and MINI show what electric cars of future will be .\nToyota confirms plans to add as many as 10 new gas/electric hybrid vehicles.'
```

For a more user-friendly dataset exploration, you can use the Hugging Face dataset hub: [news-ga-summarization](https://huggingface.co/datasets/news-ga-summarization).

✓ News retrieval (semantic search)

We will start by implementing a document retrieval system based on *semantic search*. Given a database of documents—in our case, news stories—this system takes in as input a natural language query and outputs a short list of documents matching that query.

Roughly, a semantic search system works as follows:

1. All the documents in the database are transformed and stored as *document embeddings*, that is, vectors of real numbers that represent the content of documents in an abstract, high-dimensional meaning space. (You can think of these as the document-level equivalent of the word embeddings from Project 2.)
2. When a user produces a query, the text of the query, too, is embedded in the same abstract meaning space, producing a *query embedding*.
3. The documents in the database are ranked according to the geometric (semantic) distance between their respective document embeddings and the query embedding.

We will test two retrieval methods: one with paragraph-level embeddings and one with sentence-level embeddings. These two methods vary with respect to how document embeddings are created.

Data. To develop and evaluate the news retrieval system, we will use news stories as documents and questions as queries. *Run the following cell to create a dataset class.*

```
# RetrievalDataset class
# Please execute this cell.
class RetrievalDataset:

    def __init__(self, dataset, num_documents=None):
        self.dataset = dataset

        if num_documents:
            self.num_documents = num_documents
            self.documents = self.dataset['train']['story'][:self.num_documents]
            self.queries = list(itertools.chain.from_iterable(self.dataset['train']['question'][:self.num_queries]))
            self.num_queries = len(self.queries)
        else:
            self.documents = self.dataset['train']['story']
            self.queries = list(itertools.chain.from_iterable(self.dataset['train']['question']))
            self.num_documents = len(self.documents)
            self.num_queries = len(self.queries)

        self.true_indices = []
        for i, q in enumerate(self.dataset['train']['question']):
            self.true_indices.append(i)
```

```

for i, qs in enumerate(self.dataset['train']['questions']):
    for _ in qs:
        self.true_indices.append(i)

```

Let's construct the retrieval dataset using stories as documents and questions as queries. Use only a subset of the news stories (e.g., `num_documents = 100`) as a development set for more manageable runtimes.

```
retrieval_dataset = RetrievalDataset(dataset, num_documents=100)
```

```

# view dataset
pprint(retrieval_dataset.documents[1])
print(len(retrieval_dataset.documents))
print()
print(retrieval_dataset.queries[:10])
print(retrieval_dataset.num_queries)
print("\nQueries indices:")
print(retrieval_dataset.true_indices[:20])

```

(AOL Autos) -- At the 2009 Detroit Auto Show, Chrysler, Mercedes-Benz, vehicles of the future will look like. And the future of driving looks f mph in around 4 seconds. Those in the know realize that sometime in the trucks in the US will feature electric final drive systems. The motors us batteries, fuel cells, on-board generators, and perhaps even the sun. Bu inevitability of this reality. Given our current economic times, reality ideas of what electric vehicles (or "EVs" for short) might actually look electric vehicles Chrysler showed in Detroit, Michigan, were shown at ot Each of these vehicles is a running prototype, not some pie-in-the-sky-w group of engineers at Chrysler that develops the company's EVs. To date, powered models, each quite different from the other: a Dodge Circuit EV s Country minivan (front-wheel-drive), a Jeep Wrangler Unlimited (all-whee drive). Chrysler promises to offer at least one of these models in 2010, Circuit EV photos Chrysler approaches electric vehicles with simple plug vehicles uses similar electric drive motors (only varying in power output management controller. Each plugs in to 110- or 220-volt household outle use an on-board range-extending battery charger (a generator). This gene initial batter charge has been spent (usually within a range of 40 miles vehicles an estimated range of approximately 400 miles. The generator is runs with exceptional efficiency. This technology is similar in concept to Chevrolet Volt, a vehicle that should be ready for production in 2010. A Circuit carries a larger battery pack and no generator, so its range on miles. Its large battery pack combined with compact dimensions and the e blast the car from zero-to-sixty mph in around four seconds, exceptional type. Mercedes-Benz Mercedes-Benz used the 2009 Detroit Auto Show to sho Mercedes approach was to develop one efficient body style, and then equip packages. AOL Autos: Mercedes Stirling Moss photos Much of the hardware propulsion units is built into what Mercedes calls "sandwich-floor" archi production cars. The design helps keep heavy components mounted low on t and maximized interior room. All three Concept BlueZERO vehicles include large battery pack that is said to deliver a range of 120 miles. The F-C supplements the vehicle's range with a hydrogen fuel cell. The fuel cell pack that extends cruising range to 240 miles. The E-Cell Plus, with a r champion. The key is the on-board generator powered by tiny 1-liter turb

100


```
['Where were they being deployed to?', 'What does the village train sold  
632
```

```
Queries indices:
```

```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 2, 2, 2, 2]
```

Model. To transform texts (documents or sentences) into embeddings, you will take the nowadays standard approach of using a Transformer language model. For example, you can use the [multi-qa-MiniLM-L6-cos-v1](#) Sentence Transformer.

```
# Load the language model
model_name = "multi-qa-MiniLM-L6-cos-v1"
model = SentenceTransformer(model_name)
```

```
modules.json: 100% 349/349 [00:00<00:00, 6.25kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 2.35kB/s]
README.md: 11.6k/? [00:00<00:00, 210kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 1.05kB/s]
config.json: 100% 612/612 [00:00<00:00, 10.6kB/s]
model.safetensors: 100% 90.9M/90.9M [00:02<00:00, 46.1MB/s]
tokenizer_config.json: 100% 383/383 [00:00<00:00, 5.86kB/s]
```

✓ Paragraph-level embeddings

The first method will rely on paragraph-level embeddings. That is, an entire paragraph (e.g., a whole news story) is passed as an input to a language model, the contextualised word embeddings produced by the language model for all the words in the paragraph are extracted, and the average embedding is taken to be the document representation. Luckily, the SentenceTransformer library used in the cell above automates this process for us.

First, we embed the news stories.

```
story_embeddings = model.encode(retrieval_dataset.documents)
```

```
model.encode(["Hello world"])[0][:10]

array([ 0.00667088,  0.04648961,  0.04077321,  0.00692649,  0.00061284,
        -0.06583082,  0.08488178, -0.05237267, -0.05910061,
        0.01553638],
      dtype=float32)
```

Then, we embed an example query.

```
query_embedding = model.encode('electric vehicles')
```

```
print(f"story_embeddings shape: {story_embeddings.shape}")
print(f"query_embedding shape: {query_embedding.shape}")
story_embeddings[0][:10]

story_embeddings shape: (100, 384)
query_embedding shape: (384,)
array([ 0.00789616, -0.0164791 ,  0.06144905, -0.00869662, -0.05694907,
        -0.00908399, -0.01627653, -0.10307281, -0.08313418,
        -0.04551206],
      dtype=float32)
```

```
query_embedding[:10]

array([ 0.03853272,  0.09561244, -0.02874592,  0.03049679, -0.01188052,
        0.00928078,  0.04949469, -0.01020807,  0.00878254,
        0.00132529],
      dtype=float32)
```

Now we can compute the similarity between the query and the story embeddings. We will use the dot product between embedding as a measure of their geometric distance.

Exercise: Fill in the code below by providing the right arguments to the `dot_score` function. Be careful to pick the right order of the two elements you provide.

```
# Compute semantic similarity as the dot product between embeddings
similarity_scores = util.dot_score(
    story_embeddings,
    query_embedding
)
```

Let's inspect the shape and content of the similarity score vector.

```
print(similarity_scores.shape)
print(similarity_scores)

torch.Size([100, 1])
tensor([[ -3.5164e-02],
        [ 7.4932e-01],
```

```
[ 4.2309e-01],
[ 3.4363e-01],
[ 2.5842e-01],
[ 1.6174e-01],
[ 2.6499e-01],
[ 2.1440e-01],
[ 1.2893e-01],
[ 3.4044e-01],
[ 1.5525e-01],
[ 4.4081e-01],
[ 1.7460e-01],
[ 3.7609e-02],
[ 2.3579e-01],
[ 2.3579e-01],
[ 1.6344e-01],
[ 4.0172e-01],
[ 1.2528e-01],
[ 2.2745e-01],
[ 3.6874e-01],
[ 2.8882e-01],
[ 2.2444e-01],
[ 3.4821e-01],
[ 2.1760e-01],
[ 3.3167e-01],
[ 4.9821e-01],
[ 3.8450e-01],
[ 3.9013e-01],
[ 1.5539e-01],
[ 3.8754e-01],
[ 3.4203e-01],
[ 2.8183e-01],
[ 1.9025e-01],
[ 3.4600e-01],
[ 3.4600e-01],
[ 3.2337e-01],
[-1.0555e-01],
[-4.2203e-02],
[ 3.2657e-02],
[ 1.3123e-01],
[ 6.5636e-02],
[ 6.0612e-03],
[ 8.0774e-02],
[ 2.6421e-01],
[ 1.5992e-01],
[-4.7375e-02],
[-4.7375e-02],
[-1.0589e-01],
[-1.0589e-01],
[ 9.9840e-02],
[-3.8875e-03],
[ 5.7524e-02],
[ 1.0370e-01],
[ 1.3831e-01],
[ 7.9701e-02],
[ 1.9692e-01],
[ 3.8612e-02],
```

Next, we obtain the indices of the top 3 ranked documents and their respective similarity scores. These are the documents that best match the query according to our

paragraph embeddings.

```
similarity_scores[0]
```

```
tensor([-0.0352])
```

```
similarity_scores.flatten()
```

```
tensor([-3.5164e-02,  7.4932e-01,  4.2309e-01,  3.4363e-01,
        2.5842e-01,
         1.6174e-01,  2.6499e-01,  2.1440e-01,  1.2893e-01,
        3.4044e-01,
         1.5525e-01,  4.4081e-01,  1.7460e-01,  3.7609e-02,
        2.3579e-01,
         2.3579e-01,  1.6344e-01,  4.0172e-01,  1.2528e-01,
        2.2745e-01,
         3.6874e-01,  2.8882e-01,  2.2444e-01,  3.4821e-01,
        2.1760e-01,
         3.3167e-01,  4.9821e-01,  3.8450e-01,  3.9013e-01,
        1.5539e-01,
         3.8754e-01,  3.4203e-01,  2.8183e-01,  1.9025e-01,
        3.4600e-01,
         3.4600e-01,  3.2337e-01, -1.0555e-01, -4.2203e-02,
        3.2657e-02,
         1.3123e-01,  6.5636e-02,  6.0612e-03,  8.0774e-02,
        2.6421e-01,
         1.5992e-01, -4.7375e-02, -4.7375e-02, -1.0589e-01,
       -1.0589e-01,
         9.9840e-02, -3.8875e-03,  5.7524e-02,  1.0370e-01,
        1.3831e-01,
         7.9701e-02,  1.9692e-01, -3.2610e-02,  7.1755e-04,
       -8.7851e-02,
        -5.2489e-02,  7.0033e-02,  1.3545e-02,  1.0433e-01,
        3.3496e-02,
         3.3031e-02,  1.6148e-02,  1.2311e-01,  4.0958e-02,
        8.6441e-02,
         3.3287e-03, -5.1825e-02, -1.3481e-01, -1.3887e-01,
       -1.5210e-01,
        -9.8821e-02, -9.5270e-02, -1.0855e-01, -4.1019e-02,
        2.7619e-02,
        -3.0709e-02,  1.4788e-02,  4.2993e-02,  8.7657e-03,
        3.6249e-03,
        -1.0694e-01, -3.3620e-02,  8.9359e-02,  8.9359e-02,
       -3.3313e-02,
        -1.0157e-02, -2.2729e-02, -5.6901e-02, -5.0940e-02,
        8.0547e-02,
        -1.1757e-01,  1.0669e-01, -6.1221e-02, -4.0795e-02,
       -4.0795e-02])
```

```
# Get the indices of the 3 top ranked documents and their similarity scores
top_k_scores, top_k_indices = torch.topk(similarity_scores[0], 3)
top_k_scores, top_k_indices = torch.topk(similarity_scores.flatten(), 3)
print(f'Top 3 document ids: {top_k_indices}')
print(f'Top 3 document similarity scores: {top_k_scores}')
```

```
top 3 document ids: tensor([ 1, 26, 11])
Top 3 document similarity scores: tensor([0.7493, 0.4982, 0.4408])
```

```
# Print the best matching document
print('Best matching document:\n')
best_match_index = top_k_indices[0] #picks the first element of "top_k_in
pprint(retrieval_dataset.documents[best_match_index])
```

Best matching document:

(AOL Autos) -- At the 2009 Detroit Auto Show, Chrysler, Mercedes-Benz, vehicles of the future will look like. And the future of driving looks f mph in around 4 seconds. Those in the know realize that sometime in the trucks in the US will feature electric final drive systems. The motors us batteries, fuel cells, on-board generators, and perhaps even the sun. Bu inevitability of this reality. Given our current economic times, reality ideas of what electric vehicles (or "EVs" for short) might actually look electric vehicles Chrysler showed in Detroit, Michigan, were shown at ot Each of these vehicles is a running prototype, not some pie-in-the-sky-w group of engineers at Chrysler that develops the company's EVs. To date, powered models, each quite different from the other: a Dodge Circuit EV s Country minivan (front-wheel-drive), a Jeep Wrangler Unlimited (all-whee drive). Chrysler promises to offer at least one of these models in 2010, Circuit EV photos Chrysler approaches electric vehicles with simple plug vehicles uses similar electric drive motors (only varying in power outpu management controller. Each plugs in to 110- or 220-volt household outle use an on-board range-extending battery charger (a generator). This gene initial batter charge has been spent (usually within a range of 40 miles vehicles an estimated range of approximately 400 miles. The generator is runs with exceptional efficiency. This technology is similar in concept to Chevrolet Volt, a vehicle that should be ready for production in 2010. A Circuit carries a larger battery pack and no generator, so its range on miles. Its large battery pack combined with compact dimensions and the e blast the car from zero-to-sixty mph in around four seconds, exceptional type. Mercedes-Benz Mercedes-Benz used the 2009 Detroit Auto Show to sho Mercedes approach was to develop one efficient body style, and then equip packages. AOL Autos: Mercedes Stirling Moss photos Much of the hardware propulsion units is built into what Mercedes calls "sandwich-floor" archi production cars. The design helps keep heavy components mounted low on t and maximized interior room. All three Concept BlueZERO vehicles include large battery pack that is said to deliver a range of 120 miles. The F-C supplements the vehicle's range with a hydrogen fuel cell. The fuel cell pack that extends cruising range to 240 miles. The E-Cell Plus, with a r champion. The key is the on-board generator powered by tiny 1-liter turb

✓ Putting it all together

Now that we have tested all of its components, we can reate a Python class for the document retriever based on paragraph-level document embeddings.

Exercise: Fill in the code in the cell below.

```
class ParagraphEmbeddingRetriever:

    # Initialise the Retriever object
```

```
# with a document database and a language model
def __init__(
    self,
    documents, # the list of documents in the database
    model_name, # the name of the language model
):

    # assign documents
    self.documents = documents

    # Load language model
    self.model = SentenceTransformer(
        model_name
    )

    # Embed documents
    print('Initialization: embedding documents...')
    self.doc_embeddings = self.model.encode(
        self.documents,
        batch_size=64,
        show_progress_bar=True
    )
    print(f"doc_embeddings shape: {self.doc_embeddings.shape}")

    # Retrieving function: it takes as input a query and returns
    # the top_k (default top_k = 3) matching documents in the database
    def retrieve(
        self,
        query,
        top_k=3
    ):

        # Embed query
        query_embedding = self.model.encode(
            query
        )
        print(f"query_embedding shape: {query_embedding.shape}")

        # Compute semantic similarity
        similarity_scores = util.dot_score(
            self.doc_embeddings,
            query_embedding
        )
        print(f"similarity_scores shape: {similarity_scores.shape}")
        # Get top k documents and their similarity scores
        top_k_scores, top_k_indices = torch.topk(
            similarity_scores.flatten(),
            top_k
        )
        print(f'Top {top_k} document ids: {top_k_indices}')
        print(f'Top {top_k} document similarity scores: {top_k_scores}')
        return top_k_indices, top_k_scores, similarity_scores
```

Let's test our compiled document retriever on the previous example.

```
paragraph_retriever = ParagraphEmbeddingRetriever(
    documents=retrieval_dataset.documents,
    model_name='multi-qa-MiniLM-L6-cos-v1'
)

top_3_indices, top_3_scores, _ = paragraph_retriever.retrieve(
    query='electric vehicles',
    top_k=3
)

print(f'\nTop 3 indices: {top_3_indices.tolist()}')
print(f'Their scores: {top_3_scores.tolist()}')
print('\nBest matching document:\n')

best_match_index = top_3_indices[0]
pprint(retrieval_dataset.documents[best_match_index])
```

Initialization: embedding documents...

Batches: 100%

2/2 [00:01<00:00, 1.47it/s]

doc_embeddings shape: (100, 384)

query_embedding shape: (384,)

similarity_scores shape: torch.Size([100, 1])

Top 3 document ids: tensor([1, 26, 11])

Top 3 document similarity scores: tensor([0.7493, 0.4982, 0.4408])

Top 3 indices: [1, 26, 11]

Their scores: [0.7493171691894531, 0.49820518493652344, 0.44080579280853]

Best matching document:

(AOL Autos) -- At the 2009 Detroit Auto Show, Chrysler, Mercedes-Benz, vehicles of the future will look like. And the future of driving looks f mph in around 4 seconds. Those in the know realize that sometime in the trucks in the US will feature electric final drive systems. The motors us batteries, fuel cells, on-board generators, and perhaps even the sun. Bu inevitability of this reality. Given our current economic times, reality ideas of what electric vehicles (or "EVs" for short) might actually look electric vehicles Chrysler showed in Detroit, Michigan, were shown at ot Each of these vehicles is a running prototype, not some pie-in-the-sky-w group of engineers at Chrysler that develops the company's EVs. To date, powered models, each quite different from the other: a Dodge Circuit EV s Country minivan (front-wheel-drive), a Jeep Wrangler Unlimited (all-whee drive). Chrysler promises to offer at least one of these models in 2010, Circuit EV photos Chrysler approaches electric vehicles with simple plug vehicles uses similar electric drive motors (only varying in power outpu management controller. Each plugs in to 110- or 220-volt household outle use an on-board range-extending battery charger (a generator). This gene initial batter charge has been spent (usually within a range of 40 miles vehicles an estimated range of approximately 400 miles. The generator is runs with exceptional efficiency. This technology is similar in concept to

Chevrolet Volt, a vehicle that should be ready for production in 2010. A Circuit carries a larger battery pack and no generator, so its range on miles. Its large battery pack combined with compact dimensions and the e blast the car from zero-to-sixty mph in around four seconds, exceptional type. Mercedes-Benz Mercedes-Benz used the 2009 Detroit Auto Show to sho Mercedes approach was to develop one efficient body style, and then equip packages. AOL Autos: Mercedes Stirling Moss photos Much of the hardware propulsion units is built into what Mercedes calls "sandwich-floor" archi production cars. The design helps keep heavy components mounted low on t and maximized interior room. All three Concept BlueZERO vehicles include large battery pack that is said to deliver a range of 120 miles. The F-C supplements the vehicle's range with a hydrogen fuel cell. The fuel cell pack that extends cruising range to 240 miles. The E-Cell Plus, with a r champion. The key is the on-board generator powered by tiny 1-liter turb

✓ Sentence-level embeddings

News stories consist of many sentences. When a single embedding is used to represent them all, the meaning of each sentence is diluted and the final document embedding may become uninformative.

The second method you will implement tries to address this issue by representing documents as a set of sentence-level embeddings. It is more computationally expensive, but it can potentially lead to more accurate retrieval.

The method works as follows:

1. Every document in the database is split into sentences.
2. Each sentence in a document is embedded using a language model, and all the sentence embeddings extracted for a document are stored.
3. At retrieval time, the query text is embedded and compared to all of the sentence embeddings of each document. Instead of a single similarity score for a given document, this results in a list of similarity scores, one for each sentence in the document.
4. To obtain a single number from this list of similarity scores, a summary statistic is used, such as the mean, minimum or maximum.
5. The documents are ranked based on this summary similarity score.

We can reuse the same language model as an embedding engine.

Exercise: Write a line of code to load the Sentence Transformer.

```
model_name = "multi-qa-MiniLM-L6-cos-v1"
model = SentenceTransformer(
    model_name
)
```


To split documents into sentences, we will use the [spaCy](#) library with its `sentencizer`, or *sentence segmentation* algorithm.

```
spacy = English()
spacy.add_pipe("sentencizer")

def split(text):
    """
    Splits a text into sentences using the Spacy library.

    Parameters
    -----
    text : str
        The text to be split into sentences.

    Returns
    -----
    list
        The list of sentences in the text.
    """
    processed_text = spacy(text)
    processed_sentences = processed_text.sents

    sentences = []
    for sentence in processed_sentences:
        sentences.append(sentence.text)

    return sentences
```

Let's test the sentence segmentation algorithm.

```
split('This is a first test sentence. This is the second test sentence.
      'This is a first test sentence.',
      'This is the second test sentence.',
      'Will the segmentation algorithm detect all sentences correctly?']
```

Now that the sentence segmentation is in place, let's build the sentence-based document retriever step by step.

Exercise: In the cell below, fill in the line to embed the sentences in a story.

```
# Embed news stories

story_embeddings = []

for story in tqdm(retrieval_dataset.documents):

    # Split news story into sentences
    sentences = split(story)
```

```
# Embed sentences
sentence_embeddings = model.encode(
    sentences,
    batch_size=64,
    show_progress_bar=False
)

story_embeddings.append(sentence_embeddings)
```

100%

100/100 [00:11<00:00, 19.60it/s]

story_embeddings[:2]

```
[array([[ 0.01839105, -0.01017325, -0.02490225, ...,  0.02296104,
        -0.00999908,  0.07268589],
       [-0.04309157,  0.09435556, -0.02621134, ...,  0.04956264,
        -0.01274993,  0.00806597],
       [ 0.00255312,  0.07203825,  0.02043951, ..., -0.04674088,
        -0.0116946 ,  0.03077617],
       ...,
       [-0.01564785, -0.01229415,  0.04496354, ..., -0.03079987,
         0.01913445,  0.08367103],
       [ 0.02731771,  0.03663654,  0.07021523, ..., -0.05913527,
         0.05993585,  0.08727358],
       [ 0.08606435,  0.08375411,  0.01970379, ..., -0.03052917,
        -0.02887772,  0.04253668]], dtype=float32),
 array([[ -0.07304896,  0.0147725 ,  0.01187331, ...,  0.06783794,
        -0.0032516 ,  0.0824122 ],
       [ 0.02899713,  0.04789546,  0.11675086, ..., -0.03511121,
         0.004604 ,  0.06369965],
       [ 0.07557482,  0.0138488 , -0.07906787, ...,  0.06831237,
         0.02926927,  0.08639734],
       ...,
       [ 0.00324888,  0.06812182, -0.02064399, ...,  0.03011248,
         0.03196386, -0.02675022],
       [-0.0133476 ,  0.00685598, -0.01061636, ..., -0.13657175,
         0.03225944,  0.08892463],
       [-0.05728789, -0.00967254, -0.1421575 , ...,  0.14199118,
         0.0949586 , -0.03408505]], dtype=float32)]
```

Exercise: Fill in the code to embed the query sentence. You can use the same query sentence as above, "electric vehicles".

```
# Embed test query
query_embedding = model.encode(
    'electric vehicles'
)
```

Now we can compute the similarity scores between the query embedding and the sentence embeddings. Remember: this will yield multiple similarity scores for each document one per sentence. So we will summarise this document-specific list of

document, one per sentence. So we will summarise this document specific list of similarity scores using a summary statistic. Here, we test `mean`, `min`, and `max` as summary statistics.

```
# Store the similarity scores obtained with different summary statistics
similarity_scores = {
    'mean': [],
    'max': [],
    'min': []
}

for story_sentence_embeddings in (story_embeddings):
    sentence_similarities = util.dot_score(
        query_embedding,
        story_sentence_embeddings
    )
    similarity_scores['mean'].append(sentence_similarities.mean())
    similarity_scores['min'].append(sentence_similarities.min())
    similarity_scores['max'].append(sentence_similarities.max())
```

Let's retrieve the top 3 stories and their similarity scores.

Exercise: Fill in the line below by providing a value for the parameter k .

```
top_3_similarity_scores = dict()
top_3_story_indices = dict()

for summary_statistic in ['mean', 'min', 'max']:
    top_k_scores, top_k_indices = torch.topk(
        torch.as_tensor(similarity_scores[summary_statistic]),
        k=3
    )
    top_3_similarity_scores[summary_statistic] = top_k_scores, top_k_indices
```

Let's print out the indices of the top 3 matches based on `mean` sentence similarity.

```
top_k_scores_mean, top_k_indices_mean = top_3_similarity_scores['mean']
print(f'Top 3 document ids: {top_k_indices_mean}')
print(f'Top 3 document similarity scores: {top_k_scores_mean}')
print('\nBest matching document:\n')

best_match_index = top_k_indices_mean[0]
pprint(retrieval_dataset.documents[best_match_index])

Top 3 document ids: tensor([ 1, 23, 30])
Top 3 document similarity scores: tensor([0.4343, 0.2930, 0.2875])

Best matching document:

(AOL Autos) -- At the 2009 Detroit Auto Show, Chrysler, Mercedes-Benz,
vehicles of the future will look like. And the future of driving looks f
```

mph in around 4 seconds. Those in the know realize that sometime in the trucks in the US will feature electric final drive systems. The motors use batteries, fuel cells, on-board generators, and perhaps even the sun. But the inevitability of this reality. Given our current economic times, reality ideas of what electric vehicles (or "EVs" for short) might actually look like. The electric vehicles Chrysler showed in Detroit, Michigan, were shown at Detroit. Each of these vehicles is a running prototype, not some pie-in-the-sky work group of engineers at Chrysler that develops the company's EVs. To date, the powered models, each quite different from the other: a Dodge Circuit EV is a Country minivan (front-wheel-drive), a Jeep Wrangler Unlimited (all-wheel-drive). Chrysler promises to offer at least one of these models in 2010. The Circuit EV photos Chrysler approaches electric vehicles with simple plug-in vehicles that use similar electric drive motors (only varying in power output and management controller). Each plugs in to 110- or 220-volt household outlets and uses an on-board range-extending battery charger (a generator). This generator's initial battery charge has been spent (usually within a range of 40 miles) before the vehicle's an estimated range of approximately 400 miles. The generator is run with exceptional efficiency. This technology is similar in concept to Chevrolet Volt, a vehicle that should be ready for production in 2010. The Circuit carries a larger battery pack and no generator, so its range is in the hundreds of miles. Its large battery pack combined with compact dimensions and the electric motor blast the car from zero-to-sixty mph in around four seconds, exceptional type. Mercedes-Benz used the 2009 Detroit Auto Show to show its approach. Mercedes approach was to develop one efficient body style, and then equip it with different packages. AOL Autos: Mercedes Stirling Moss photos Much of the hardware for the propulsion units is built into what Mercedes calls "sandwich-floor" architecture for production cars. The design helps keep heavy components mounted low on the chassis and maximized interior room. All three Concept BlueZERO vehicles include a large battery pack that is said to deliver a range of 120 miles. The F-Cell supplements the vehicle's range with a hydrogen fuel cell. The fuel cell pack that extends cruising range to 240 miles. The E-Cell Plus, with a range of 160 miles, is the champion. The key is the on-board generator powered by tiny 1-liter turbodiesel engines.

Exercise: In the next two cells, write code to inspect the top 3 indices and their scores as well as the best matching document for the `min` and `max` summary statistics.

```
top_k_scores_min, top_k_indices_min = top_3_similarity_scores['min']

print(f'Top 3 document ids: {top_k_indices_min}')
print(f'Top 3 document similarity scores: {top_k_scores_min}')
print('\nBest matching document:\n')

best_match_index = top_k_indices_min[0]
pprint(retrieval_dataset.documents[best_match_index])
```

```
Top 3 document ids: tensor([11, 30, 31])
Top 3 document similarity scores: tensor([0.0815, 0.0447, 0.0440])
```

Best matching document:

(AOL Autos) -- Consumer Reports recently announced its annual used cars report. The major categories was "Best in Fuel Economy." With gasoline and oil prices rising, that's a key factor these days when choosing a used car -- or even a new car. According to Consumer Reports, "the affordable and reliable vehicles [that passed] the real-world fuel-economy tests." What follows is a list of the vehicles in the under \$10,000 price range, with a short description of each vehicle.

the under \$10,000 price range, with a short description of each vehicle. calculated by Consumer Reports in their own on-the-road tests. 2000 Honda hit showrooms in 1999 for the 2000 model year, it was hailed as one of the best in the world. It was the first gasoline-electric hybrid to be sold in the U.S. and won several technology awards. The Insight employs Honda's Integrated Motor Assist (IMA) lightweight 1.0-liter, three-cylinder gasoline automobile engine with an automatic transmission, but a continuously variable transmission (CVT). 2002 Toyota Prius Overall MPG: 41 This was one of the earliest general purpose cars that certainly has become a green living icon since then. Drawing its power from a 1.8-liter engine and an electric motor, these earlier editions generated only 70 hp. It's pod-like design had a certain retro-futuristic appeal, and was roomy and comfortable. Echo Overall MPG: 38 The Echo was rolled out in 2000 as a no-frills compact car, but it was on a budget that precluded it from snagging a Corolla. It was reliable and efficient, and the taut suspension delivers a ride comparable to a Corolla. Comes with a 1.5-liter four-cylinder 108-hp 16-Valve EFI engine. 1998 GM/Toyota plant. A front-wheel-drive subcompact sedan, the Prizm was initially marketed as a GM model, but later became a Chevy model. Available as a base model and LS, the Prizm had engines that produced 120 hp in 1998 and 1999, but the addition of VVT-i in 2000 improved its performance. Protégé LX Overall MPG: 32 Mazda redesigned the Protégé in 1995, upgrading its engine and much-improved ride quality -- not to mention increased interior space and crisp handling and solid performance for a small car. This model year has become a classic. 1998 - 2000 Toyota Corolla CE/LE Overall MPG: 32 More than 40 years, for its quality, dependability and, maybe most importantly, its reliability. Editions are powered by a 1.8 L, 120-hp, four-cylinder 16-Valve EFI, with the 1998 edition tweaked the engine to deliver 125 ponies. A front-wheel drive job that was reliable and efficient, but Consumer Reports chose the midline CE and top-end LE. 1998 Acura Integra sport coupe and sport sedan

```
top_k_scores_max, top_k_indices_max = top_3_similarity_scores['max']

print(f'Top 3 document ids: {top_k_indices_max}')
print(f'Top 3 document similarity scores: {top_k_scores_max}')
print('\nBest matching document:\n')

best_match_index = top_k_indices_max[0]
pprint(retrieval_dataset.documents[best_match_index])
```

```
Top 3 document ids: tensor([ 1, 26,  2])
Top 3 document similarity scores: tensor([0.7618, 0.5786, 0.4867])
```

Best matching document:

(AOL Autos) -- At the 2009 Detroit Auto Show, Chrysler, Mercedes-Benz, and other manufacturers showed off vehicles of the future will look like. And the future of driving looks fast. Some of the vehicles shown at the show can accelerate from 0 to 60 mph in around 4 seconds. Those in the know realize that sometime in the future, all cars and trucks in the US will feature electric final drive systems. The motors use lithium-ion batteries, fuel cells, on-board generators, and perhaps even the sun. But the inevitability of this reality. Given our current economic times, reality is that we need to find ideas of what electric vehicles (or "EVs" for short) might actually look like. Some of the electric vehicles Chrysler showed in Detroit, Michigan, were shown at the show. Each of these vehicles is a running prototype, not some pie-in-the-sky-wish. A group of engineers at Chrysler that develops the company's EVs. To date, they have shown several powered models, each quite different from the other: a Dodge Circuit EV is a compact car, a Country minivan (front-wheel-drive), a Jeep Wrangler Unlimited (all-wheel-drive). Chrysler promises to offer at least one of these models in 2010. The Circuit EV photos Chrysler approaches electric vehicles with simple plug-in hybrids. The vehicles use similar electric drive motors (only varying in power output).

management controller. Each plugs in to 110- or 220-volt household outlet and use an on-board range-extending battery charger (a generator). This gives the initial battery charge has been spent (usually within a range of 40 miles). The vehicles have an estimated range of approximately 400 miles. The generator is run with exceptional efficiency. This technology is similar in concept to the Chevrolet Volt, a vehicle that should be ready for production in 2010. A Circuit carries a larger battery pack and no generator, so its range is 100 miles. Its large battery pack combined with compact dimensions and the electric motor blast the car from zero-to-sixty mph in around four seconds, exceptional type. Mercedes-Benz used the 2009 Detroit Auto Show to show its approach was to develop one efficient body style, and then equip it with different packages. AOL Autos: Mercedes Stirling Moss photos Much of the hardware for the propulsion units is built into what Mercedes calls "sandwich-floor" architecture for production cars. The design helps keep heavy components mounted low on the chassis and maximized interior room. All three Concept BlueZERO vehicles include a large battery pack that is said to deliver a range of 120 miles. The F-Cell supplements the vehicle's range with a hydrogen fuel cell. The fuel cell pack that extends cruising range to 240 miles. The E-Cell Plus, with a range of 150 miles, is the champion. The key is the on-board generator powered by tiny 1-liter turbo

Do the different strategies produce different results?

2025-10-18: ALR - yes but mean and max was the same text

✓ Putting it all together

Now that we have tested all of its components, we can create a Python class for the document retriever based on on sentence-level document embeddings.

Fill in the code in the cell below.

```
# SentenceEmbeddingRetriever class

class SentenceEmbeddingRetriever:

    # Initialise the Retriever object
    # with a document database and a language model
    def __init__(
        self,
        documents,
        model_name,
    ):

        # Load language model
        self.model = SentenceTransformer(
            model_name
        )

        # Load the sentence segmentation algorithm
        self.spacy = English()
        self.spacy.add_pipe("sentencizer")
```

```
# Define function to split document into sentences
def _split(document):
    return [sent.text for sent in self.spacy(document).sents]

# Embed documents
self.document_embeddings = []
print('Initialization: embedding documents...')
for document in tqdm(documents):

    # Split document into sentences
    sentences = _split(
        document
    )

    # Embed sentences
    sentence_embeddings = self.model.encode(
        sentences,
        batch_size=64,
        show_progress_bar=False
    )

    self.document_embeddings.append(sentence_embeddings)
#print(f"\nFormed document_embeddings of len {len(self.document_emb

# Retrieving function: it takes as input a query and returns
# the top_k (default top_k = 3) matching documents in the database
def retrieve(
    self,
    query,
    mode,
    top_k=3
):

    # Embed query
    query_embedding = self.model.encode(
        query
    )
    #print(f"\nMade query_embedding of len {len(query_embedding)}")

    # Compute sentence-level similarity scores
    similarity_scores = []
    for doc_sentence_embeddings in self.document_embeddings:
        sentence_similarities = util.dot_score(
            doc_sentence_embeddings,
            query_embedding
        )

        if mode == 'max':
            similarity_scores.append(
                sentence_similarities.max()
            )
        elif mode == 'mean':
            similarity_scores.append(
                sentence_similarities.mean()
```

```

    )
    elif mode == 'min':
        similarity_scores.append(
            sentence_similarities.min()
        )
    #print(f"Calculated similarity_scores of len {len(similarity_scores)}")

    # Transform scores into a tensor, for compatibility
    similarity_scores = torch.as_tensor(similarity_scores)

    # Get top k documents and their similarity scores
    top_k_scores, top_k_indices = torch.topk(
        torch.as_tensor(similarity_scores),
        k=top_k
    )
    return top_k_indices, top_k_scores, similarity_scores

```

Now let's test it on the previous example. Use `mode=mean` to define the summary statistic in the `retrieve` function.

Exercise: Fill in the code in the cell below.

```

sentence_retriever = SentenceEmbeddingRetriever(
    documents=retrieval_dataset.documents,
    model_name='multi-qa-MiniLM-L6-cos-v1'
)

top_3_indices, top_3_scores, all_similarity_scores = sentence_retriever
    query='electric vehicles',
    mode='mean'
)

print(f'\nTop 3 indices: {top_3_indices.tolist()}')
print(f'Scores: {top_3_scores.tolist()}')
print('\nBest matching document:\n')

best_match_index = top_3_indices[0]
pprint(retrieval_dataset.documents[best_match_index])

```

Initialization: embedding documents...

100%

100/100 [00:02<00:00, 50.37it/s]

Top 3 indices: [1, 23, 30]

Scores: [0.43427833914756775, 0.29296183586120605, 0.2874869406223297]

Best matching document:

(AOL Autos) -- At the 2009 Detroit Auto Show, Chrysler, Mercedes-Benz, vehicles of the future will look like. And the future of driving looks f mph in around 4 seconds. Those in the know realize that sometime in the

trucks in the US will feature electric final drive systems. The motors use batteries, fuel cells, on-board generators, and perhaps even the sun. But the inevitability of this reality. Given our current economic times, reality ideas of what electric vehicles (or "EVs" for short) might actually look like. Electric vehicles Chrysler showed in Detroit, Michigan, were shown at each of these vehicles is a running prototype, not some pie-in-the-sky group of engineers at Chrysler that develops the company's EVs. To date, powered models, each quite different from the other: a Dodge Circuit EV is a Country minivan (front-wheel-drive), a Jeep Wrangler Unlimited (all-wheel-drive). Chrysler promises to offer at least one of these models in 2010. Circuit EV photos Chrysler approaches electric vehicles with simple plug-in vehicles use similar electric drive motors (only varying in power output management controller. Each plugs in to 110- or 220-volt household outlet use an on-board range-extending battery charger (a generator). This generation initial battery charge has been spent (usually within a range of 40 miles) vehicles an estimated range of approximately 400 miles. The generator is runs with exceptional efficiency. This technology is similar in concept to Chevrolet Volt, a vehicle that should be ready for production in 2010. A Circuit carries a larger battery pack and no generator, so its range on miles. Its large battery pack combined with compact dimensions and the electric blast the car from zero-to-sixty mph in around four seconds, exceptional type. Mercedes-Benz Mercedes-Benz used the 2009 Detroit Auto Show to show Mercedes approach was to develop one efficient body style, and then equip packages. AOL Autos: Mercedes Stirling Moss photos Much of the hardware propulsion units is built into what Mercedes calls "sandwich-floor" architecture production cars. The design helps keep heavy components mounted low on the and maximized interior room. All three Concept BlueZERO vehicles include large battery pack that is said to deliver a range of 120 miles. The F-Cell supplements the vehicle's range with a hydrogen fuel cell. The fuel cell pack that extends cruising range to 240 miles. The E-Cell Plus, with a range champion. The key is the on-board generator powered by tiny 1-liter turbodiesel.

✓ Evaluation

How to tell whether the sentence-based retriever actually works better than the paragraph-based one? And how to pick the best summary statistic within the sentence-based approach?

To assess and compare models, we will use *accuracy@k* (or *top-k accuracy*) as an evaluation metric. This metric computes the number of times when the correct document is among the top k documents retrieved. This is a generalized version of accuracy, with *accuracy@1* being equivalent to the standard notion of accuracy (how often the correct document is the top document retrieved). We use it because in practice, in a retrieval scenario, you most likely want to return multiple hits to the user (think of a Google search) so it can be considered sufficient for the target document to be in the top k .

How do we know what the target document for a query is? Remember that our dataset includes news, questions, answers, and summaries. To evaluate the news retrieval

includes news, questions, answers, and summaries. To evaluate the news retrieval system, we will use news stories as target documents and take their respective questions as queries.

For each value of `k` in `[1, 3, 5, 10]` and for each model type in `[paragraph, sentence_mean, sentence_min, sentence_max]`, we will compute the top-`k` accuracy of that model type and store it for convenience in the following data structure (i.e., you will be replacing all the zeros with actual accuracy scores).

```
top_k_accuracy = {
    1: {"paragraph": 0, "sentence_mean": 0, "sentence_min": 0, "senten
    3: {"paragraph": 0, "sentence_mean": 0, "sentence_min": 0, "senten
    5: {"paragraph": 0, "sentence_mean": 0, "sentence_min": 0, "senten
    10: {"paragraph": 0, "sentence_mean": 0, "sentence_min": 0, "senten
}
```

Let's start from the paragraph-based model.

```
paragraph_retriever = ParagraphEmbeddingRetriever(
    documents=retrieval_dataset.documents,
    model_name='multi-qa-MiniLM-L6-cos-v1'
)

# Collect similarity scores for all the queries in the dataset
# (so we can measure top k accuracy for any k we want)
all_similarity_scores = []

for query in retrieval_dataset.queries:
    _, _, similarity_scores = paragraph_retriever.retrieve(
        query=query
    )
    all_similarity_scores.append(similarity_scores)

# Convert the list of tensors into a single tensor
# added due to error ValueError: Found array with dim 3. None expected
all_similarity_scores = torch.stack(all_similarity_scores).squeeze(-1)
print(f"Shape of all_similarity_scores: {all_similarity_scores.shape}")

# Now compute accuracy for each value of k
for k in [1, 3, 5, 10]:
    acc = top_k_accuracy_score(
        retrieval_dataset.true_indices, # these are the target document
        all_similarity_scores,
        k=k
    )

    top_k_accuracy[k]["paragraph"] = acc
```

Initialization: embedding documents...

Batches: 100%

2/2 [00:00<00:00, 2.78it/s]

```
doc_embeddings shape: (100, 384)
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([73, 97, 0])
Top 3 document similarity scores: tensor([0.2526, 0.2357, 0.2269])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 0, 79, 75])
Top 3 document similarity scores: tensor([0.4919, 0.1600, 0.1593])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 0, 40, 45])
Top 3 document similarity scores: tensor([0.4929, 0.1940, 0.1928])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 0, 54, 73])
Top 3 document similarity scores: tensor([0.4363, 0.1784, 0.1725])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 0, 18, 81])
Top 3 document similarity scores: tensor([0.3988, 0.1758, 0.1706])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 0, 75, 76])
Top 3 document similarity scores: tensor([0.4261, 0.2403, 0.2221])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 0, 75, 93])
Top 3 document similarity scores: tensor([0.5146, 0.2051, 0.1755])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([39, 47, 46])
Top 3 document similarity scores: tensor([0.2920, 0.2619, 0.2619])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 0, 48, 49])
Top 3 document similarity scores: tensor([0.5537, 0.2416, 0.2416])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 0, 49, 48])
Top 3 document similarity scores: tensor([0.4975, 0.1779, 0.1779])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 0, 75, 86])
Top 3 document similarity scores: tensor([0.4918, 0.2248, 0.1891])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 0, 49, 48])
Top 3 document similarity scores: tensor([0.5266, 0.2808, 0.2808])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 0, 73, 81])
```

```
Top 3 document similarity scores: tensor([0.6043, 0.2736, 0.2027])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3,  1, 28])
Top 3 document similarity scores: tensor([0.3678, 0.3667, 0.3376])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 1, 27,  3])
Top 3 document similarity scores: tensor([0.6367, 0.4122, 0.4096])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([1, 6, 3])
Top 3 document similarity scores: tensor([0.3725, 0.3666, 0.3227])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([93, 29, 39])
Top 3 document similarity scores: tensor([0.2736, 0.2607, 0.2606])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 2, 11, 28])
Top 3 document similarity scores: tensor([0.5404, 0.4617, 0.4518])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 2,  9, 32])
Top 3 document similarity scores: tensor([0.5370, 0.5145, 0.5076])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 2, 17,  9])
Top 3 document similarity scores: tensor([0.5598, 0.4658, 0.4597])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([22, 28, 21])
Top 3 document similarity scores: tensor([0.3067, 0.2494, 0.2411])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([22, 28, 21])
Top 3 document similarity scores: tensor([0.3593, 0.3110, 0.2897])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 2, 32,  9])
Top 3 document similarity scores: tensor([0.5192, 0.4675, 0.4472])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 2,  9, 17])
Top 3 document similarity scores: tensor([0.5386, 0.4458, 0.3942])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3, 27, 25])
Top 3 document similarity scores: tensor([0.5209, 0.4540, 0.4337])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([3, 2, 4])
Top 3 document similarity scores: tensor([0.4033, 0.3939, 0.3620])
```

```
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3, 30,  9])
Top 3 document similarity scores: tensor([0.3556, 0.3504, 0.3269])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([28,  3, 26])
Top 3 document similarity scores: tensor([0.3362, 0.3243, 0.3068])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([52, 32, 61])
Top 3 document similarity scores: tensor([0.2760, 0.2122, 0.2076])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3, 28, 30])
Top 3 document similarity scores: tensor([0.6116, 0.5144, 0.5048])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3, 30, 27])
Top 3 document similarity scores: tensor([0.4306, 0.4080, 0.4036])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3, 25, 27])
Top 3 document similarity scores: tensor([0.5235, 0.4583, 0.4515])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3, 27, 28])
Top 3 document similarity scores: tensor([0.5930, 0.5146, 0.5061])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([63, 22,  4])
Top 3 document similarity scores: tensor([0.2628, 0.2535, 0.2503])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([28,  3, 11])
Top 3 document similarity scores: tensor([0.4741, 0.4301, 0.4054])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 75, 79])
Top 3 document similarity scores: tensor([0.3528, 0.2855, 0.2800])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 4, 22,  2])
Top 3 document similarity scores: tensor([0.5850, 0.3361, 0.3141])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([77, 76, 74])
Top 3 document similarity scores: tensor([0.3283, 0.3214, 0.2895])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 4, 96, 48])
Top 3 document similarity scores: tensor([0.1765, 0.1426, 0.1195])
query_embedding shape: (384,)
```

```
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([76, 77, 75])
Top 3 document similarity scores: tensor([0.3776, 0.3711, 0.3494])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([74, 75, 76])
Top 3 document similarity scores: tensor([0.3720, 0.3392, 0.2763])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 5, 31, 26])
Top 3 document similarity scores: tensor([0.5690, 0.4023, 0.3866])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3, 61, 64])
Top 3 document similarity scores: tensor([0.1379, 0.1377, 0.1267])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([54, 43, 29])
Top 3 document similarity scores: tensor([0.2970, 0.2344, 0.2303])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 5, 31, 26])
Top 3 document similarity scores: tensor([0.6743, 0.4158, 0.3953])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 5, 28, 31])
Top 3 document similarity scores: tensor([0.6527, 0.4315, 0.4010])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 5, 31, 26])
Top 3 document similarity scores: tensor([0.6490, 0.4252, 0.4184])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 5, 31, 28])
Top 3 document similarity scores: tensor([0.6796, 0.4537, 0.4131])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 5, 31, 11])
Top 3 document similarity scores: tensor([0.6622, 0.4486, 0.4146])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 6,  5, 52])
Top 3 document similarity scores: tensor([0.4181, 0.2754, 0.2666])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 6, 52,  5])
Top 3 document similarity scores: tensor([0.4078, 0.2652, 0.2313])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([86, 97,  6])
Top 3 document similarity scores: tensor([0.4573, 0.3445, 0.2677])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
```

```
Top 3 document ids: tensor([ 6,  2, 35])
Top 3 document similarity scores: tensor([0.7158, 0.5109, 0.5047])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 7,  2, 23])
Top 3 document similarity scores: tensor([0.7086, 0.4284, 0.4182])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 8,  7, 19])
Top 3 document similarity scores: tensor([0.3083, 0.2639, 0.2255])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 7,  8, 51])
Top 3 document similarity scores: tensor([0.2849, 0.2432, 0.2126])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 7,  8, 84])
Top 3 document similarity scores: tensor([0.3142, 0.2449, 0.1822])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 7,  8, 51])
Top 3 document similarity scores: tensor([0.2557, 0.2045, 0.1970])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([23, 33, 12])
Top 3 document similarity scores: tensor([0.4912, 0.4573, 0.3871])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([33, 12, 16])
Top 3 document similarity scores: tensor([0.2347, 0.2212, 0.1750])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 7,  8, 51])
Top 3 document similarity scores: tensor([0.3002, 0.2619, 0.2036])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([23, 33, 12])
Top 3 document similarity scores: tensor([0.5027, 0.4871, 0.3829])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 7, 51,  8])
Top 3 document similarity scores: tensor([0.2430, 0.2307, 0.1862])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([19,  9, 39])
Top 3 document similarity scores: tensor([0.2059, 0.1522, 0.1461])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([20, 22, 40])
Top 3 document similarity scores: tensor([0.3088, 0.2510, 0.2405])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([27,  9, 26])
```

```
Top 3 document similarity scores: tensor([0.2694, 0.2633, 0.2591])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([10,  6, 28])
Top 3 document similarity scores: tensor([0.4798, 0.4402, 0.3452])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([10,  9, 82])
Top 3 document similarity scores: tensor([0.3447, 0.2825, 0.2171])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([10,  6,  2])
Top 3 document similarity scores: tensor([0.6592, 0.4442, 0.3509])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([99, 98, 58])
Top 3 document similarity scores: tensor([0.2201, 0.2201, 0.1931])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 6, 10, 24])
Top 3 document similarity scores: tensor([0.4721, 0.4682, 0.3228])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([10,  6, 35])
Top 3 document similarity scores: tensor([0.4061, 0.3935, 0.3710])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([47, 46, 81])
Top 3 document similarity scores: tensor([0.2355, 0.2355, 0.2340])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([35, 34, 27])
Top 3 document similarity scores: tensor([0.4747, 0.4747, 0.4679])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([27, 72, 88])
Top 3 document similarity scores: tensor([0.2755, 0.2464, 0.2278])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([11, 26,  1])
Top 3 document similarity scores: tensor([0.6126, 0.5219, 0.4837])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3, 11,  9])
Top 3 document similarity scores: tensor([0.2965, 0.2854, 0.2713])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([11, 26, 31])
Top 3 document similarity scores: tensor([0.5892, 0.5301, 0.4843])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([27, 11, 30])
Top 3 document similarity scores: tensor([0.4501, 0.4457, 0.3438])
```



```
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([11, 26, 31])
Top 3 document similarity scores: tensor([0.5853, 0.5313, 0.4825])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([27,  3, 21])
Top 3 document similarity scores: tensor([0.4584, 0.4119, 0.4000])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 6, 64, 44])
Top 3 document similarity scores: tensor([0.2787, 0.2567, 0.2225])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([12, 33, 29])
Top 3 document similarity scores: tensor([0.6622, 0.3698, 0.3593])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([12, 33,  7])
Top 3 document similarity scores: tensor([0.5870, 0.3634, 0.3190])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([12, 86, 33])
Top 3 document similarity scores: tensor([0.4020, 0.3772, 0.3544])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([97, 86, 16])
Top 3 document similarity scores: tensor([0.1874, 0.1704, 0.1659])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([12, 33, 29])
Top 3 document similarity scores: tensor([0.6142, 0.3521, 0.3466])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([12, 86, 33])
Top 3 document similarity scores: tensor([0.5460, 0.3480, 0.3384])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([12, 33, 29])
Top 3 document similarity scores: tensor([0.4530, 0.3648, 0.3478])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([44, 22,  1])
Top 3 document similarity scores: tensor([0.2826, 0.2741, 0.2383])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([12, 33, 29])
Top 3 document similarity scores: tensor([0.6670, 0.3488, 0.3455])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([13, 36, 17])
Top 3 document similarity scores: tensor([0.4744, 0.2761, 0.2549])
query_embedding shape: (384,)
```

```
- - -
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([13, 35, 34])
Top 3 document similarity scores: tensor([0.6502, 0.3315, 0.3315])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([13, 17, 36])
Top 3 document similarity scores: tensor([0.6406, 0.3647, 0.3077])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([13, 36, 22])
Top 3 document similarity scores: tensor([0.5318, 0.3187, 0.2619])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([72, 34, 35])
Top 3 document similarity scores: tensor([0.1626, 0.1588, 0.1588])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([58, 61, 96])
Top 3 document similarity scores: tensor([0.2791, 0.2768, 0.2714])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([25, 20, 22])
Top 3 document similarity scores: tensor([0.6306, 0.5294, 0.5277])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([13, 22, 7])
Top 3 document similarity scores: tensor([0.3568, 0.3263, 0.2640])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 6, 24, 15])
Top 3 document similarity scores: tensor([0.4706, 0.4700, 0.4611])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([14, 15, 20])
Top 3 document similarity scores: tensor([0.3456, 0.3456, 0.2842])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([25, 2, 27])
Top 3 document similarity scores: tensor([0.6433, 0.5452, 0.5160])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([24, 6, 14])
Top 3 document similarity scores: tensor([0.4786, 0.4785, 0.4675])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([16, 45, 25])
Top 3 document similarity scores: tensor([0.5810, 0.3409, 0.2626])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([16, 45, 36])
Top 3 document similarity scores: tensor([0.6678, 0.3671, 0.3351])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
```

```

Top 3 document ids: tensor([16, 45, 4])
Top 3 document similarity scores: tensor([0.6229, 0.3826, 0.2873])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([16, 45, 4])
Top 3 document similarity scores: tensor([0.6557, 0.3449, 0.2522])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([60, 57, 66])
Top 3 document similarity scores: tensor([0.2819, 0.2622, 0.2130])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([25, 17, 36])
Top 3 document similarity scores: tensor([0.3061, 0.2690, 0.1500])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([17, 36, 25])
Top 3 document similarity scores: tensor([0.4572, 0.3634, 0.3000])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([25, 17, 33])
Top 3 document similarity scores: tensor([0.4877, 0.4020, 0.3349])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([18, 24, 88])
Top 3 document similarity scores: tensor([0.3360, 0.1986, 0.1798])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([18, 22, 41])
Top 3 document similarity scores: tensor([0.2275, 0.1701, 0.1374])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([18, 97, 86])
Top 3 document similarity scores: tensor([0.4810, 0.2549, 0.2419])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([18, 34, 35])
Top 3 document similarity scores: tensor([0.5451, 0.3194, 0.3194])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([18, 97, 90])
Top 3 document similarity scores: tensor([0.5274, 0.2956, 0.2854])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([19, 25, 12])
Top 3 document similarity scores: tensor([0.5370, 0.2611, 0.2475])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([74, 79, 23])
Top 3 document similarity scores: tensor([0.1844, 0.1841, 0.1751])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([19, 25, 14])

```

```
-
Top 3 document similarity scores: tensor([0.3009, 0.2959, 0.2890])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([20, 40, 22])
Top 3 document similarity scores: tensor([0.2721, 0.2359, 0.2121])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([65, 44, 62])
Top 3 document similarity scores: tensor([0.3688, 0.3171, 0.2768])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([40, 56, 22])
Top 3 document similarity scores: tensor([0.2595, 0.2334, 0.2162])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([20,  2, 28])
Top 3 document similarity scores: tensor([0.6185, 0.4132, 0.4044])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([36,  1, 17])
Top 3 document similarity scores: tensor([0.4961, 0.4254, 0.4077])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([27, 26, 36])
Top 3 document similarity scores: tensor([0.4973, 0.4666, 0.4356])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 2, 22, 20])
Top 3 document similarity scores: tensor([0.4333, 0.4236, 0.4097])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([22,  7, 33])
Top 3 document similarity scores: tensor([0.4023, 0.3467, 0.2688])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3, 22, 28])
Top 3 document similarity scores: tensor([0.5878, 0.5444, 0.5404])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 8,  7, 23])
Top 3 document similarity scores: tensor([0.2961, 0.2686, 0.2553])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([22,  9,  2])
Top 3 document similarity scores: tensor([0.5861, 0.4046, 0.4022])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 2, 22,  6])
Top 3 document similarity scores: tensor([0.4426, 0.4209, 0.4041])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([90, 45, 19])
Top 3 document similarity scores: tensor([0.1743, 0.1647, 0.1466])
```

```
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([90, 23, 33])
Top 3 document similarity scores: tensor([0.4651, 0.4203, 0.3708])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([23, 3, 14])
Top 3 document similarity scores: tensor([0.3569, 0.3186, 0.3018])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([90, 45, 19])
Top 3 document similarity scores: tensor([0.1812, 0.1701, 0.1512])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([24, 30, 3])
Top 3 document similarity scores: tensor([0.2982, 0.2838, 0.2753])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([24, 88, 87])
Top 3 document similarity scores: tensor([0.6049, 0.4663, 0.4663])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([87, 88, 24])
Top 3 document similarity scores: tensor([0.2590, 0.2590, 0.2564])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([24, 53, 39])
Top 3 document similarity scores: tensor([0.3342, 0.2859, 0.2787])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([88, 87, 24])
Top 3 document similarity scores: tensor([0.4994, 0.4994, 0.4340])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([87, 88, 90])
Top 3 document similarity scores: tensor([0.3048, 0.3048, 0.2543])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([27, 3, 30])
Top 3 document similarity scores: tensor([0.5652, 0.5255, 0.4905])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([20, 22, 2])
Top 3 document similarity scores: tensor([0.5317, 0.5200, 0.5138])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([25, 3, 17])
Top 3 document similarity scores: tensor([0.4248, 0.3848, 0.3457])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([2, 3, 27])
Top 3 document similarity scores: tensor([0.6060, 0.6033, 0.5825])
query_embedding shape: (384,)
```

```
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([19, 1, 17])
Top 3 document similarity scores: tensor([0.2447, 0.2220, 0.1816])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([25, 17, 1])
Top 3 document similarity scores: tensor([0.6033, 0.4465, 0.4118])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([25, 17, 3])
Top 3 document similarity scores: tensor([0.5013, 0.4079, 0.3556])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([31, 11, 26])
Top 3 document similarity scores: tensor([0.6180, 0.5931, 0.5053])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([79, 61, 76])
Top 3 document similarity scores: tensor([0.2352, 0.2024, 0.1969])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([26, 28, 61])
Top 3 document similarity scores: tensor([0.4929, 0.3445, 0.3422])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([26, 11, 44])
Top 3 document similarity scores: tensor([0.4373, 0.3193, 0.2943])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([26, 75, 82])
Top 3 document similarity scores: tensor([0.3436, 0.2185, 0.2084])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([31, 26, 11])
Top 3 document similarity scores: tensor([0.5751, 0.5412, 0.5133])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([26, 31, 5])
Top 3 document similarity scores: tensor([0.5926, 0.4233, 0.3559])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 4, 23, 76])
Top 3 document similarity scores: tensor([0.1549, 0.1536, 0.1121])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([27, 11, 24])
Top 3 document similarity scores: tensor([0.5121, 0.3298, 0.3195])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([27, 25, 23])
Top 3 document similarity scores: tensor([0.4778, 0.3621, 0.3550])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
```

```
Top 3 document ids: tensor([ 4, 76, 23])
Top 3 document similarity scores: tensor([0.1571, 0.1237, 0.1129])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([24, 27,  3])
Top 3 document similarity scores: tensor([0.4392, 0.4163, 0.3690])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([27,  3, 17])
Top 3 document similarity scores: tensor([0.5250, 0.4445, 0.3602])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([27, 24,  6])
Top 3 document similarity scores: tensor([0.5149, 0.3817, 0.3747])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([53, 88, 87])
Top 3 document similarity scores: tensor([0.2374, 0.2302, 0.2302])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([28,  2,  3])
Top 3 document similarity scores: tensor([0.6805, 0.6026, 0.5590])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([28, 21, 30])
Top 3 document similarity scores: tensor([0.4519, 0.4144, 0.3851])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([28,  3, 27])
Top 3 document similarity scores: tensor([0.6940, 0.6408, 0.6149])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([36, 30, 17])
Top 3 document similarity scores: tensor([0.5165, 0.4935, 0.4879])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3, 28, 27])
Top 3 document similarity scores: tensor([0.6267, 0.6262, 0.5967])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 5, 11, 26])
Top 3 document similarity scores: tensor([0.5299, 0.4964, 0.4800])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([29, 12, 25])
Top 3 document similarity scores: tensor([0.4638, 0.4521, 0.4307])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([58, 37, 59])
Top 3 document similarity scores: tensor([0.3041, 0.2880, 0.2853])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([29, 18, 33])
```

```
Top 3 document similarity scores: tensor([0.4830, 0.3572, 0.3061])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([29, 12, 33])
Top 3 document similarity scores: tensor([0.6805, 0.3067, 0.2990])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([29, 17, 33])
Top 3 document similarity scores: tensor([0.5532, 0.2716, 0.2700])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([27,  3, 28])
Top 3 document similarity scores: tensor([0.5936, 0.5866, 0.5426])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 2,  3, 27])
Top 3 document similarity scores: tensor([0.6055, 0.5937, 0.5862])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 2, 65, 32])
Top 3 document similarity scores: tensor([0.2815, 0.2696, 0.2538])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 2, 30,  3])
Top 3 document similarity scores: tensor([0.6344, 0.6147, 0.5539])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 2, 22, 30])
Top 3 document similarity scores: tensor([0.6923, 0.5445, 0.5367])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3,  2, 27])
Top 3 document similarity scores: tensor([0.5759, 0.5526, 0.5522])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([30,  3, 28])
Top 3 document similarity scores: tensor([0.5461, 0.5380, 0.5092])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 2, 32, 30])
Top 3 document similarity scores: tensor([0.6650, 0.5100, 0.5091])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 2,  3, 27])
Top 3 document similarity scores: tensor([0.5924, 0.5190, 0.5160])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([75, 37, 71])
Top 3 document similarity scores: tensor([0.3263, 0.2837, 0.2801])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3, 28, 27])
Top 3 document similarity scores: tensor([0.5424, 0.5284, 0.5194])
```



```
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 7, 31, 28])
Top 3 document similarity scores: tensor([0.4689, 0.3848, 0.3805])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 6, 24, 15])
Top 3 document similarity scores: tensor([0.4283, 0.3869, 0.3485])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([31, 26, 11])
Top 3 document similarity scores: tensor([0.5495, 0.4379, 0.4375])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([75, 79, 77])
Top 3 document similarity scores: tensor([0.2870, 0.2510, 0.2305])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([32, 20,  2])
Top 3 document similarity scores: tensor([0.6276, 0.3574, 0.3449])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([32, 20,  2])
Top 3 document similarity scores: tensor([0.6399, 0.4795, 0.4739])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([32,  6,  2])
Top 3 document similarity scores: tensor([0.5692, 0.3348, 0.3268])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([75, 79, 78])
Top 3 document similarity scores: tensor([0.3412, 0.2988, 0.2726])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([35, 34, 18])
Top 3 document similarity scores: tensor([0.2579, 0.2579, 0.2192])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([33, 12,  6])
Top 3 document similarity scores: tensor([0.4502, 0.3686, 0.3281])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([18, 63, 35])
Top 3 document similarity scores: tensor([0.2333, 0.2220, 0.1969])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 37, 75])
Top 3 document similarity scores: tensor([0.2399, 0.2368, 0.2301])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([25,  2, 14])
Top 3 document similarity scores: tensor([0.4627, 0.4280, 0.4274])
query_embedding shape: (384,)
```

```
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([33, 27, 12])
Top 3 document similarity scores: tensor([0.4213, 0.3522, 0.3506])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([34, 35, 11])
Top 3 document similarity scores: tensor([0.5414, 0.5414, 0.3372])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([34, 35, 27])
Top 3 document similarity scores: tensor([0.5701, 0.5701, 0.4246])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([51, 63, 54])
Top 3 document similarity scores: tensor([0.2276, 0.1639, 0.1572])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([34, 35, 11])
Top 3 document similarity scores: tensor([0.5414, 0.5414, 0.3372])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([32, 40, 67])
Top 3 document similarity scores: tensor([0.2808, 0.2644, 0.2338])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 3, 27,  2])
Top 3 document similarity scores: tensor([0.5828, 0.5298, 0.5190])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([36, 60, 19])
Top 3 document similarity scores: tensor([0.1545, 0.1453, 0.1313])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([36, 17,  1])
Top 3 document similarity scores: tensor([0.5967, 0.4974, 0.3086])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([44, 32,  9])
Top 3 document similarity scores: tensor([0.3434, 0.3067, 0.2878])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([36, 17,  1])
Top 3 document similarity scores: tensor([0.5873, 0.5021, 0.3003])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([24, 18,  6])
Top 3 document similarity scores: tensor([0.2868, 0.2862, 0.2348])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([37, 59, 60])
Top 3 document similarity scores: tensor([0.3026, 0.2742, 0.2629])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
```

```
Top 3 document ids: tensor([37, 24, 64])
Top 3 document similarity scores: tensor([0.3293, 0.1665, 0.1433])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([37, 59, 60])
Top 3 document similarity scores: tensor([0.3528, 0.3284, 0.3217])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([37, 24, 59])
Top 3 document similarity scores: tensor([0.2746, 0.1983, 0.1733])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 6, 24, 86])
Top 3 document similarity scores: tensor([0.2424, 0.2421, 0.1624])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 6, 86, 24])
Top 3 document similarity scores: tensor([0.3048, 0.2764, 0.2324])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([57, 38, 67])
Top 3 document similarity scores: tensor([0.5359, 0.4728, 0.3175])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([17, 13,  1])
Top 3 document similarity scores: tensor([0.2045, 0.1332, 0.1008])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([38, 58, 60])
Top 3 document similarity scores: tensor([0.4660, 0.2883, 0.2692])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([57, 38, 37])
Top 3 document similarity scores: tensor([0.2005, 0.1722, 0.1514])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([32,  5, 40])
Top 3 document similarity scores: tensor([0.3417, 0.2862, 0.2809])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([57, 38, 67])
Top 3 document similarity scores: tensor([0.6205, 0.5208, 0.2672])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([57, 38, 29])
Top 3 document similarity scores: tensor([0.4492, 0.3950, 0.2842])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([38, 57, 66])
Top 3 document similarity scores: tensor([0.6630, 0.5486, 0.2940])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([38, 57, 67])
```

```
Top 3 document similarity scores: tensor([0.6545, 0.5231, 0.2976])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([55, 42, 54])
Top 3 document similarity scores: tensor([0.2793, 0.2532, 0.2518])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([55, 39, 93])
Top 3 document similarity scores: tensor([0.4345, 0.4077, 0.3561])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([42, 39, 54])
Top 3 document similarity scores: tensor([0.5230, 0.4522, 0.3583])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([39, 80, 54])
Top 3 document similarity scores: tensor([0.4486, 0.3893, 0.3515])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([55, 88, 87])
Top 3 document similarity scores: tensor([0.3581, 0.3270, 0.3270])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([39, 55, 42])
Top 3 document similarity scores: tensor([0.4899, 0.4352, 0.3662])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([55, 54, 42])
Top 3 document similarity scores: tensor([0.2398, 0.2389, 0.2060])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([40, 52, 22])
Top 3 document similarity scores: tensor([0.6191, 0.5138, 0.3598])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([40, 52, 22])
Top 3 document similarity scores: tensor([0.5476, 0.3039, 0.2643])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 6, 12, 78])
Top 3 document similarity scores: tensor([0.3517, 0.3148, 0.2609])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 6, 12, 33])
Top 3 document similarity scores: tensor([0.2940, 0.2683, 0.2518])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([52, 40, 22])
Top 3 document similarity scores: tensor([0.4615, 0.3947, 0.3799])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([40, 52, 22])
Top 3 document similarity scores: tensor([0.6789, 0.4313, 0.2736])
```

```
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([40, 56, 52])
Top 3 document similarity scores: tensor([0.6132, 0.2922, 0.2802])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([41, 59, 60])
Top 3 document similarity scores: tensor([0.2883, 0.2639, 0.2423])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([41, 59, 60])
Top 3 document similarity scores: tensor([0.5315, 0.3238, 0.2939])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([41, 60, 59])
Top 3 document similarity scores: tensor([0.5436, 0.3268, 0.3040])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([41, 64, 37])
Top 3 document similarity scores: tensor([0.2390, 0.1836, 0.1519])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([41, 58, 52])
Top 3 document similarity scores: tensor([0.4976, 0.2684, 0.2478])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([41, 37, 50])
Top 3 document similarity scores: tensor([0.2517, 0.1542, 0.1306])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([41, 81, 27])
Top 3 document similarity scores: tensor([0.3909, 0.2158, 0.1929])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([62, 64, 65])
Top 3 document similarity scores: tensor([0.2427, 0.2178, 0.2127])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([42, 39, 46])
Top 3 document similarity scores: tensor([0.6764, 0.3914, 0.3343])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([54, 55, 39])
Top 3 document similarity scores: tensor([0.2831, 0.2235, 0.2188])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([42, 39, 53])
Top 3 document similarity scores: tensor([0.7004, 0.3783, 0.2977])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([42, 53, 39])
Top 3 document similarity scores: tensor([0.5247, 0.4125, 0.3613])
query_embedding shape: (384,)
```

```

similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([42, 53, 39])
Top 3 document similarity scores: tensor([0.6360, 0.3911, 0.3763])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([42, 53, 46])
Top 3 document similarity scores: tensor([0.3938, 0.3519, 0.3047])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([53, 42, 50])
Top 3 document similarity scores: tensor([0.4271, 0.4156, 0.4088])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([42, 39, 53])
Top 3 document similarity scores: tensor([0.6740, 0.3710, 0.3115])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([43, 69, 54])
Top 3 document similarity scores: tensor([0.4803, 0.2311, 0.2235])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([43, 13, 47])
Top 3 document similarity scores: tensor([0.2232, 0.2133, 0.2074])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([50, 56, 39])
Top 3 document similarity scores: tensor([0.4901, 0.4692, 0.3637])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([43, 35, 34])
Top 3 document similarity scores: tensor([0.4948, 0.2601, 0.2601])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([43, 34, 35])
Top 3 document similarity scores: tensor([0.5342, 0.2679, 0.2679])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([43, 34, 35])
Top 3 document similarity scores: tensor([0.4772, 0.2962, 0.2962])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([44, 50, 54])
Top 3 document similarity scores: tensor([0.3848, 0.2885, 0.2832])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([44, 26, 5])
Top 3 document similarity scores: tensor([0.4973, 0.2826, 0.2788])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([44, 1, 2])
Top 3 document similarity scores: tensor([0.2839, 0.1720, 0.1388])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])

```

```
Top 3 document ids: tensor([45, 56, 90])
Top 3 document similarity scores: tensor([0.5303, 0.3089, 0.2965])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([45, 56, 90])
Top 3 document similarity scores: tensor([0.5468, 0.3855, 0.3333])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([11, 3, 26])
Top 3 document similarity scores: tensor([0.3099, 0.2977, 0.2809])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([45, 56, 23])
Top 3 document similarity scores: tensor([0.2895, 0.2266, 0.2249])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([67, 82, 64])
Top 3 document similarity scores: tensor([0.2284, 0.2220, 0.2068])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([80, 36, 48])
Top 3 document similarity scores: tensor([0.2184, 0.2096, 0.1909])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([47, 46, 50])
Top 3 document similarity scores: tensor([0.4374, 0.4374, 0.3730])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([50, 56, 53])
Top 3 document similarity scores: tensor([0.4798, 0.4559, 0.3642])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([46, 47, 50])
Top 3 document similarity scores: tensor([0.4843, 0.4843, 0.4435])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([46, 47, 80])
Top 3 document similarity scores: tensor([0.3232, 0.3232, 0.2286])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([46, 47, 56])
Top 3 document similarity scores: tensor([0.4399, 0.4399, 0.4212])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([50, 46, 47])
Top 3 document similarity scores: tensor([0.3856, 0.3842, 0.3842])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([50, 53, 47])
Top 3 document similarity scores: tensor([0.4558, 0.4477, 0.4095])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([50, 56, 53])
```

```
Top 3 document similarity scores: tensor([0.5127, 0.5084, 0.4073])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([49, 48, 51])
Top 3 document similarity scores: tensor([0.5447, 0.5447, 0.3370])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([48, 49, 51])
Top 3 document similarity scores: tensor([0.6210, 0.6210, 0.2838])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([48, 49, 80])
Top 3 document similarity scores: tensor([0.3561, 0.3561, 0.3061])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([49, 48, 41])
Top 3 document similarity scores: tensor([0.3187, 0.3187, 0.2835])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([49, 48, 39])
Top 3 document similarity scores: tensor([0.5215, 0.5215, 0.2980])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([96, 56, 50])
Top 3 document similarity scores: tensor([0.2756, 0.2493, 0.2453])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([39, 50, 53])
Top 3 document similarity scores: tensor([0.3459, 0.3371, 0.3257])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([49, 48, 51])
Top 3 document similarity scores: tensor([0.3068, 0.3068, 0.2129])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([49, 48, 51])
Top 3 document similarity scores: tensor([0.5915, 0.5915, 0.2783])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([39, 50, 53])
Top 3 document similarity scores: tensor([0.5293, 0.5188, 0.5007])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([54, 56, 50])
Top 3 document similarity scores: tensor([0.4649, 0.3939, 0.3688])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([56, 50, 53])
Top 3 document similarity scores: tensor([0.5759, 0.5328, 0.4449])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([49, 48, 51])
Top 3 document similarity scores: tensor([0.4001, 0.4001, 0.3747])
```



```
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([51, 55, 42])
Top 3 document similarity scores: tensor([0.2438, 0.2014, 0.1851])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([51, 43, 48])
Top 3 document similarity scores: tensor([0.4057, 0.3100, 0.1899])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([50, 53, 51])
Top 3 document similarity scores: tensor([0.3770, 0.3461, 0.3184])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([48, 49, 51])
Top 3 document similarity scores: tensor([0.4028, 0.4028, 0.3580])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([51, 66, 49])
Top 3 document similarity scores: tensor([0.2949, 0.2026, 0.1336])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([51, 54, 47])
Top 3 document similarity scores: tensor([0.2657, 0.2170, 0.1935])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([46, 47, 81])
Top 3 document similarity scores: tensor([0.2104, 0.2104, 0.2093])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([52, 80, 46])
Top 3 document similarity scores: tensor([0.6590, 0.3141, 0.2700])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([52,  5, 80])
Top 3 document similarity scores: tensor([0.5543, 0.2354, 0.2251])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([52, 51, 96])
Top 3 document similarity scores: tensor([0.5241, 0.2343, 0.2074])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([52,  5, 61])
Top 3 document similarity scores: tensor([0.4983, 0.2370, 0.2328])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([52, 40, 80])
Top 3 document similarity scores: tensor([0.4239, 0.3942, 0.2913])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([51, 81, 44])
Top 3 document similarity scores: tensor([0.1811, 0.1584, 0.1508])
query_embedding shape: (384,)
```

```

similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([56, 53, 50])
Top 3 document similarity scores: tensor([0.4279, 0.4041, 0.3883])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([56, 50, 53])
Top 3 document similarity scores: tensor([0.5930, 0.5109, 0.5092])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([53, 56, 47])
Top 3 document similarity scores: tensor([0.6374, 0.4795, 0.4283])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([53, 55, 47])
Top 3 document similarity scores: tensor([0.4646, 0.2811, 0.2683])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([53, 50, 56])
Top 3 document similarity scores: tensor([0.3729, 0.3603, 0.3383])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([54,  0, 53])
Top 3 document similarity scores: tensor([0.3474, 0.3351, 0.3302])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([54, 18, 35])
Top 3 document similarity scores: tensor([0.3296, 0.1789, 0.1707])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([54, 52, 56])
Top 3 document similarity scores: tensor([0.4620, 0.2046, 0.1810])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([54, 43, 50])
Top 3 document similarity scores: tensor([0.4175, 0.3027, 0.2540])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([56, 50, 53])
Top 3 document similarity scores: tensor([0.5089, 0.4110, 0.4088])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([56, 50, 53])
Top 3 document similarity scores: tensor([0.4947, 0.4668, 0.4456])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([56, 49, 48])
Top 3 document similarity scores: tensor([0.2316, 0.1827, 0.1827])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([57, 38, 52])
Top 3 document similarity scores: tensor([0.3974, 0.2911, 0.2500])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])

```

```

Top 3 document ids: tensor([57, 38, 67])
Top 3 document similarity scores: tensor([0.4391, 0.2445, 0.1856])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([57, 38, 37])
Top 3 document similarity scores: tensor([0.4200, 0.3058, 0.2775])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([57, 38, 61])
Top 3 document similarity scores: tensor([0.4960, 0.3369, 0.2648])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([57, 38, 64])
Top 3 document similarity scores: tensor([0.4502, 0.3207, 0.2701])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([34, 35, 6])
Top 3 document similarity scores: tensor([0.2360, 0.2360, 0.2241])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([57, 38, 37])
Top 3 document similarity scores: tensor([0.4921, 0.3290, 0.2844])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([34, 35, 6])
Top 3 document similarity scores: tensor([0.3212, 0.3212, 0.2696])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([59, 64, 6])
Top 3 document similarity scores: tensor([0.2635, 0.2415, 0.2397])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([58, 75, 71])
Top 3 document similarity scores: tensor([0.3790, 0.2829, 0.2729])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([58, 75, 72])
Top 3 document similarity scores: tensor([0.3628, 0.2762, 0.2676])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([58, 41, 20])
Top 3 document similarity scores: tensor([0.4784, 0.3078, 0.2557])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([58, 60, 44])
Top 3 document similarity scores: tensor([0.3053, 0.2797, 0.2722])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([59, 60, 41])
Top 3 document similarity scores: tensor([0.5952, 0.5821, 0.2858])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([59, 57, 51])

```

```
Top 3 document similarity scores: tensor([0.4265, 0.2261, 0.2050])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([59, 57, 60])
Top 3 document similarity scores: tensor([0.5484, 0.2310, 0.1889])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([60, 59, 58])
Top 3 document similarity scores: tensor([0.4966, 0.4942, 0.3174])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([59, 60, 58])
Top 3 document similarity scores: tensor([0.5813, 0.5449, 0.2473])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([59, 60, 58])
Top 3 document similarity scores: tensor([0.5356, 0.5321, 0.2770])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([59, 60, 41])
Top 3 document similarity scores: tensor([0.6493, 0.6253, 0.2487])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([81, 15, 14])
Top 3 document similarity scores: tensor([0.1844, 0.1786, 0.1786])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([37, 58, 59])
Top 3 document similarity scores: tensor([0.2609, 0.2344, 0.2189])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([60, 59, 57])
Top 3 document similarity scores: tensor([0.5478, 0.4904, 0.1834])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([60, 41, 54])
Top 3 document similarity scores: tensor([0.4289, 0.2620, 0.1726])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([60, 59, 75])
Top 3 document similarity scores: tensor([0.4456, 0.3396, 0.2513])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([60, 41, 54])
Top 3 document similarity scores: tensor([0.3741, 0.2914, 0.1863])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([61, 15, 14])
Top 3 document similarity scores: tensor([0.3575, 0.2816, 0.2816])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([61, 57, 68])
Top 3 document similarity scores: tensor([0.5075, 0.3220, 0.2917])
```

```

query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([61, 41, 27])
Top 3 document similarity scores: tensor([0.6868, 0.2804, 0.2796])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([61, 69, 41])
Top 3 document similarity scores: tensor([0.7134, 0.3028, 0.2804])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([61, 57, 68])
Top 3 document similarity scores: tensor([0.5738, 0.3143, 0.2928])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([61, 68, 65])
Top 3 document similarity scores: tensor([0.6109, 0.2882, 0.2809])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([62, 60, 52])
Top 3 document similarity scores: tensor([0.3690, 0.2512, 0.2243])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([62, 52, 58])
Top 3 document similarity scores: tensor([0.5851, 0.3598, 0.3477])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([62, 52, 48])
Top 3 document similarity scores: tensor([0.5486, 0.3503, 0.3314])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([62, 49, 48])
Top 3 document similarity scores: tensor([0.5370, 0.3130, 0.3130])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([62, 88, 87])
Top 3 document similarity scores: tensor([0.4415, 0.3247, 0.3247])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([62, 58, 59])
Top 3 document similarity scores: tensor([0.6004, 0.3263, 0.3206])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([62, 58, 60])
Top 3 document similarity scores: tensor([0.4081, 0.2691, 0.2391])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([62, 58, 59])
Top 3 document similarity scores: tensor([0.4931, 0.3294, 0.3195])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([62, 87, 88])
Top 3 document similarity scores: tensor([0.4973, 0.2989, 0.2989])
query_embedding shape: (384,)

```

```

similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([62, 58, 59])
Top 3 document similarity scores: tensor([0.5794, 0.2837, 0.2773])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([63, 18, 50])
Top 3 document similarity scores: tensor([0.2500, 0.2497, 0.1881])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([63, 35, 34])
Top 3 document similarity scores: tensor([0.5037, 0.2774, 0.2774])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([35, 34, 61])
Top 3 document similarity scores: tensor([0.2832, 0.2832, 0.2827])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([63, 35, 34])
Top 3 document similarity scores: tensor([0.3572, 0.3329, 0.3329])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([63, 18, 54])
Top 3 document similarity scores: tensor([0.5757, 0.2407, 0.1809])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([63, 57, 80])
Top 3 document similarity scores: tensor([0.2804, 0.1494, 0.1451])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([64, 65, 67])
Top 3 document similarity scores: tensor([0.5740, 0.3249, 0.2773])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 64, 23])
Top 3 document similarity scores: tensor([0.2960, 0.2439, 0.2439])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([64, 61, 2])
Top 3 document similarity scores: tensor([0.4386, 0.2765, 0.2441])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 64, 23])
Top 3 document similarity scores: tensor([0.3069, 0.2560, 0.2469])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([64, 32, 9])
Top 3 document similarity scores: tensor([0.4596, 0.2367, 0.1727])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([64, 32, 2])
Top 3 document similarity scores: tensor([0.5289, 0.2139, 0.1563])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])

```

```

Top 3 document ids: tensor([64, 85, 95])
Top 3 document similarity scores: tensor([0.3897, 0.2800, 0.2648])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([64, 57, 67])
Top 3 document similarity scores: tensor([0.5802, 0.3506, 0.2753])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([64, 65, 67])
Top 3 document similarity scores: tensor([0.5502, 0.3347, 0.2760])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([65, 37, 64])
Top 3 document similarity scores: tensor([0.5295, 0.3342, 0.2530])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([35, 34, 65])
Top 3 document similarity scores: tensor([0.3431, 0.3431, 0.3376])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([65, 67, 56])
Top 3 document similarity scores: tensor([0.4231, 0.3469, 0.2598])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([65, 67, 70])
Top 3 document similarity scores: tensor([0.5369, 0.3622, 0.2670])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([40, 65, 20])
Top 3 document similarity scores: tensor([0.2756, 0.1953, 0.1267])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([40, 32, 20])
Top 3 document similarity scores: tensor([0.3709, 0.3001, 0.2878])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([66, 56, 28])
Top 3 document similarity scores: tensor([0.6413, 0.2433, 0.2108])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([64, 66, 53])
Top 3 document similarity scores: tensor([0.2862, 0.2700, 0.1797])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([64, 66, 61])
Top 3 document similarity scores: tensor([0.3152, 0.2756, 0.1721])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([57, 67, 61])
Top 3 document similarity scores: tensor([0.2467, 0.2049, 0.1913])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([66, 92, 91])

```

```
Top 3 document similarity scores: tensor([0.3093, 0.1318, 0.1218])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([66, 56, 28])
Top 3 document similarity scores: tensor([0.6346, 0.2301, 0.2190])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([28, 67, 61])
Top 3 document similarity scores: tensor([0.2026, 0.2003, 0.1819])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([64, 57, 67])
Top 3 document similarity scores: tensor([0.4396, 0.3630, 0.3597])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([61, 44, 67])
Top 3 document similarity scores: tensor([0.2564, 0.2489, 0.2407])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([67, 70, 61])
Top 3 document similarity scores: tensor([0.5269, 0.3174, 0.2220])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([67, 61, 57])
Top 3 document similarity scores: tensor([0.5585, 0.3706, 0.3405])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([57, 67, 64])
Top 3 document similarity scores: tensor([0.4398, 0.4275, 0.4237])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([72, 73, 60])
Top 3 document similarity scores: tensor([0.2327, 0.2169, 0.2138])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([67, 70, 65])
Top 3 document similarity scores: tensor([0.4136, 0.4092, 0.2138])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([67, 70, 61])
Top 3 document similarity scores: tensor([0.6265, 0.3852, 0.3718])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([67, 57, 61])
Top 3 document similarity scores: tensor([0.5848, 0.3549, 0.2790])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([68, 69, 61])
Top 3 document similarity scores: tensor([0.6043, 0.2271, 0.2079])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([68, 61, 62])
Top 3 document similarity scores: tensor([0.5482, 0.2901, 0.2001])
```



```
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([68, 61, 70])
Top 3 document similarity scores: tensor([0.6104, 0.3086, 0.2110])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([68, 57, 61])
Top 3 document similarity scores: tensor([0.5944, 0.2518, 0.2490])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([57, 37, 59])
Top 3 document similarity scores: tensor([0.3094, 0.2966, 0.2310])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([69, 61, 41])
Top 3 document similarity scores: tensor([0.6441, 0.3091, 0.2704])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([69, 61, 57])
Top 3 document similarity scores: tensor([0.6017, 0.3468, 0.2372])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([69, 59, 37])
Top 3 document similarity scores: tensor([0.4980, 0.2279, 0.2100])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([69, 61, 66])
Top 3 document similarity scores: tensor([0.5949, 0.2794, 0.2126])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([69, 17, 41])
Top 3 document similarity scores: tensor([0.4187, 0.2712, 0.2618])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([10, 5, 89])
Top 3 document similarity scores: tensor([0.2065, 0.1804, 0.1685])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([70, 67, 61])
Top 3 document similarity scores: tensor([0.6994, 0.3210, 0.2364])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([61, 56, 64])
Top 3 document similarity scores: tensor([0.3244, 0.2883, 0.2854])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([68, 61, 70])
Top 3 document similarity scores: tensor([0.3514, 0.3429, 0.3404])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([70, 67, 65])
Top 3 document similarity scores: tensor([0.4065, 0.3483, 0.2178])
query_embedding shape: (384,)
```

```
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([70, 61, 67])
Top 3 document similarity scores: tensor([0.5450, 0.2605, 0.2595])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([71, 76, 77])
Top 3 document similarity scores: tensor([0.4692, 0.3019, 0.2713])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([17, 65, 60])
Top 3 document similarity scores: tensor([0.2284, 0.2222, 0.2140])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([76, 77, 71])
Top 3 document similarity scores: tensor([0.3551, 0.3472, 0.3374])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([71, 76, 75])
Top 3 document similarity scores: tensor([0.4477, 0.2673, 0.2386])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([71, 76, 78])
Top 3 document similarity scores: tensor([0.6432, 0.3876, 0.3805])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([72, 34, 35])
Top 3 document similarity scores: tensor([0.2312, 0.2229, 0.2229])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([14, 15, 79])
Top 3 document similarity scores: tensor([0.2656, 0.2656, 0.2435])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([71, 6, 78])
Top 3 document similarity scores: tensor([0.5792, 0.3045, 0.2700])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([71, 76, 77])
Top 3 document similarity scores: tensor([0.4425, 0.2614, 0.2376])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([71, 76, 77])
Top 3 document similarity scores: tensor([0.4349, 0.2858, 0.2721])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([89, 13, 14])
Top 3 document similarity scores: tensor([0.3168, 0.2346, 0.2287])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([72, 78, 81])
Top 3 document similarity scores: tensor([0.2528, 0.2058, 0.1822])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
```

```
Top 3 document ids: tensor([75, 60, 78])
Top 3 document similarity scores: tensor([0.2778, 0.2336, 0.2126])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([72, 78, 76])
Top 3 document similarity scores: tensor([0.6393, 0.1986, 0.1898])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([72, 78, 76])
Top 3 document similarity scores: tensor([0.6249, 0.2699, 0.2538])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([73, 72, 81])
Top 3 document similarity scores: tensor([0.6224, 0.2704, 0.2326])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([73, 72, 81])
Top 3 document similarity scores: tensor([0.6318, 0.3016, 0.2148])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([73, 72, 81])
Top 3 document similarity scores: tensor([0.5903, 0.2723, 0.2385])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([72, 81, 73])
Top 3 document similarity scores: tensor([0.3488, 0.3393, 0.3172])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([73, 72, 52])
Top 3 document similarity scores: tensor([0.3685, 0.2274, 0.2094])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([79, 74, 78])
Top 3 document similarity scores: tensor([0.3030, 0.2573, 0.2536])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([74, 76, 71])
Top 3 document similarity scores: tensor([0.2518, 0.2460, 0.2402])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([23, 78, 74])
Top 3 document similarity scores: tensor([0.2643, 0.2418, 0.2266])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([75, 82, 72])
Top 3 document similarity scores: tensor([0.4364, 0.2093, 0.2018])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([49, 48, 54])
Top 3 document similarity scores: tensor([0.3558, 0.3558, 0.2715])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([49, 48, 51])
```

```
Top 3 document similarity scores: tensor([0.3158, 0.3158, 0.2360])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([75, 84, 89])
Top 3 document similarity scores: tensor([0.3701, 0.2433, 0.2393])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([10, 6, 37])
Top 3 document similarity scores: tensor([0.2547, 0.2108, 0.1992])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([49, 48, 51])
Top 3 document similarity scores: tensor([0.3141, 0.3141, 0.2304])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([10, 37, 58])
Top 3 document similarity scores: tensor([0.2644, 0.2555, 0.2508])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([75, 86, 89])
Top 3 document similarity scores: tensor([0.3569, 0.2736, 0.2252])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([6, 75, 58])
Top 3 document similarity scores: tensor([0.2731, 0.2527, 0.2187])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([49, 48, 52])
Top 3 document similarity scores: tensor([0.3560, 0.3560, 0.2483])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([66, 51, 48])
Top 3 document similarity scores: tensor([0.1927, 0.1638, 0.1568])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([76, 77, 75])
Top 3 document similarity scores: tensor([0.3267, 0.3144, 0.2160])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([76, 77, 71])
Top 3 document similarity scores: tensor([0.5151, 0.4871, 0.4471])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([77, 76, 72])
Top 3 document similarity scores: tensor([0.3113, 0.2606, 0.2583])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([77, 76, 43])
Top 3 document similarity scores: tensor([0.3845, 0.3211, 0.1942])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([76, 78, 77])
Top 3 document similarity scores: tensor([0.4345, 0.3895, 0.3803])
```

```
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([77, 76, 43])
Top 3 document similarity scores: tensor([0.3847, 0.3516, 0.1581])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([77, 76, 72])
Top 3 document similarity scores: tensor([0.6114, 0.5706, 0.3364])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([77, 76, 43])
Top 3 document similarity scores: tensor([0.4099, 0.3738, 0.1510])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([77, 76, 72])
Top 3 document similarity scores: tensor([0.5832, 0.5279, 0.3197])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([77, 76, 72])
Top 3 document similarity scores: tensor([0.3035, 0.2549, 0.2031])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([77, 76, 72])
Top 3 document similarity scores: tensor([0.5791, 0.5155, 0.3244])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([77, 76, 72])
Top 3 document similarity scores: tensor([0.4393, 0.3834, 0.2146])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([77, 76, 72])
Top 3 document similarity scores: tensor([0.6412, 0.5780, 0.3587])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([78, 74, 71])
Top 3 document similarity scores: tensor([0.4602, 0.4342, 0.4222])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([78, 76, 71])
Top 3 document similarity scores: tensor([0.4499, 0.4270, 0.4045])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 79, 86])
Top 3 document similarity scores: tensor([0.3882, 0.3170, 0.2833])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([79, 75, 71])
Top 3 document similarity scores: tensor([0.3349, 0.2684, 0.2415])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([79, 71, 75])
Top 3 document similarity scores: tensor([0.3664, 0.3342, 0.3006])
query_embedding shape: (384,)
```

```
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([60, 74, 38])
Top 3 document similarity scores: tensor([0.2828, 0.1921, 0.1795])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([71, 75, 79])
Top 3 document similarity scores: tensor([0.3523, 0.3456, 0.3199])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([71, 79, 75])
Top 3 document similarity scores: tensor([0.4769, 0.4262, 0.4225])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([71, 75, 74])
Top 3 document similarity scores: tensor([0.3123, 0.2880, 0.2764])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([79, 71, 75])
Top 3 document similarity scores: tensor([0.3893, 0.3428, 0.3376])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([80, 48, 49])
Top 3 document similarity scores: tensor([0.5558, 0.2597, 0.2597])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([26, 39, 52])
Top 3 document similarity scores: tensor([0.1206, 0.1201, 0.1193])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([80, 55, 50])
Top 3 document similarity scores: tensor([0.3559, 0.3472, 0.3250])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([42, 80, 50])
Top 3 document similarity scores: tensor([0.2920, 0.2841, 0.2829])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([52, 81, 61])
Top 3 document similarity scores: tensor([0.3482, 0.2598, 0.2499])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([81, 80, 50])
Top 3 document similarity scores: tensor([0.6161, 0.2160, 0.1814])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([81, 99, 98])
Top 3 document similarity scores: tensor([0.5090, 0.1342, 0.1342])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([81, 18, 87])
Top 3 document similarity scores: tensor([0.4443, 0.1704, 0.1522])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
```

```

Top 3 document ids: tensor([81, 80, 87])
Top 3 document similarity scores: tensor([0.5791, 0.1896, 0.1841])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([81, 52, 50])
Top 3 document similarity scores: tensor([0.3454, 0.3355, 0.3218])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([81, 50, 52])
Top 3 document similarity scores: tensor([0.3658, 0.3318, 0.3096])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([82, 61, 83])
Top 3 document similarity scores: tensor([0.2204, 0.2096, 0.1993])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([87, 88, 92])
Top 3 document similarity scores: tensor([0.2084, 0.2084, 0.1682])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([81, 52, 60])
Top 3 document similarity scores: tensor([0.2396, 0.2204, 0.2175])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([83, 82, 90])
Top 3 document similarity scores: tensor([0.2245, 0.2229, 0.2002])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([ 1, 17, 29])
Top 3 document similarity scores: tensor([0.2759, 0.2608, 0.2411])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([82, 83, 79])
Top 3 document similarity scores: tensor([0.7249, 0.5543, 0.2603])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([82, 84, 93])
Top 3 document similarity scores: tensor([0.4139, 0.3325, 0.3179])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([82, 83, 90])
Top 3 document similarity scores: tensor([0.6483, 0.5673, 0.2572])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([89, 82, 97])
Top 3 document similarity scores: tensor([0.2726, 0.2637, 0.2472])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([82, 83, 84])
Top 3 document similarity scores: tensor([0.7491, 0.4651, 0.2878])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([82, 84, 83])

```

```
Top 3 document similarity scores: tensor([0.5963, 0.3525, 0.3220])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([82, 83, 79])
Top 3 document similarity scores: tensor([0.7232, 0.5370, 0.2090])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([83, 44, 17])
Top 3 document similarity scores: tensor([0.3279, 0.2323, 0.2127])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([83, 82, 79])
Top 3 document similarity scores: tensor([0.4344, 0.3436, 0.2004])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([83, 82, 79])
Top 3 document similarity scores: tensor([0.4582, 0.3884, 0.2065])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([15, 14, 64])
Top 3 document similarity scores: tensor([0.2653, 0.2653, 0.2637])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 58, 86])
Top 3 document similarity scores: tensor([0.2945, 0.2805, 0.2679])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([75, 74, 64])
Top 3 document similarity scores: tensor([0.3044, 0.2981, 0.2847])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 89, 93])
Top 3 document similarity scores: tensor([0.5392, 0.3677, 0.3415])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 8, 93])
Top 3 document similarity scores: tensor([0.3889, 0.2918, 0.2809])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 97, 92])
Top 3 document similarity scores: tensor([0.4950, 0.4568, 0.4450])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 82, 89])
Top 3 document similarity scores: tensor([0.4491, 0.3393, 0.3229])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 55, 39])
Top 3 document similarity scores: tensor([0.2918, 0.2384, 0.2232])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 89, 93])
Top 3 document similarity scores: tensor([0.4034, 0.1905, 0.1842])
```



```

query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 93, 91])
Top 3 document similarity scores: tensor([0.4360, 0.3301, 0.2864])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 89, 91])
Top 3 document similarity scores: tensor([0.5136, 0.2300, 0.2255])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([81, 99, 98])
Top 3 document similarity scores: tensor([0.2962, 0.2528, 0.2528])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 89, 82])
Top 3 document similarity scores: tensor([0.4602, 0.3511, 0.3485])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 89, 82])
Top 3 document similarity scores: tensor([0.4623, 0.3478, 0.3315])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 93, 91])
Top 3 document similarity scores: tensor([0.4436, 0.3459, 0.2960])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 39, 55])
Top 3 document similarity scores: tensor([0.2446, 0.1753, 0.1637])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 93, 91])
Top 3 document similarity scores: tensor([0.4363, 0.3372, 0.3156])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 89, 93])
Top 3 document similarity scores: tensor([0.4034, 0.1905, 0.1842])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 89, 92])
Top 3 document similarity scores: tensor([0.4574, 0.2398, 0.2370])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 92, 97])
Top 3 document similarity scores: tensor([0.4311, 0.4180, 0.3569])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 92, 93])
Top 3 document similarity scores: tensor([0.5337, 0.3863, 0.3035])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([37, 85, 58])
Top 3 document similarity scores: tensor([0.2396, 0.2206, 0.2009])
query_embedding shape: (384,)

```

```
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 86, 54])
Top 3 document similarity scores: tensor([0.2541, 0.1716, 0.1685])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 82, 90])
Top 3 document similarity scores: tensor([0.3359, 0.2592, 0.2545])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 92, 89])
Top 3 document similarity scores: tensor([0.3342, 0.3037, 0.2751])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([86, 97, 85])
Top 3 document similarity scores: tensor([0.4968, 0.4958, 0.3958])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([86, 97, 99])
Top 3 document similarity scores: tensor([0.4009, 0.2597, 0.2095])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([86, 97, 85])
Top 3 document similarity scores: tensor([0.5242, 0.4843, 0.4021])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([86, 92, 77])
Top 3 document similarity scores: tensor([0.4405, 0.2107, 0.1838])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([86, 97, 85])
Top 3 document similarity scores: tensor([0.4292, 0.2584, 0.2518])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([86, 97, 6])
Top 3 document similarity scores: tensor([0.6036, 0.3715, 0.2643])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([90, 86, 92])
Top 3 document similarity scores: tensor([0.3678, 0.3013, 0.2631])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([97, 86, 85])
Top 3 document similarity scores: tensor([0.5000, 0.4963, 0.3949])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 92, 88])
Top 3 document similarity scores: tensor([0.3634, 0.3017, 0.2710])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([87, 88, 24])
Top 3 document similarity scores: tensor([0.4015, 0.4015, 0.1966])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
```

```

Top 3 document ids: tensor([44, 50, 32])
Top 3 document similarity scores: tensor([0.1453, 0.1379, 0.1355])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 95, 97])
Top 3 document similarity scores: tensor([0.2996, 0.2555, 0.2241])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([97, 86, 85])
Top 3 document similarity scores: tensor([0.4617, 0.3972, 0.3060])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([88, 87, 94])
Top 3 document similarity scores: tensor([0.3498, 0.3498, 0.2665])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([87, 88, 99])
Top 3 document similarity scores: tensor([0.2931, 0.2931, 0.2312])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 86, 95])
Top 3 document similarity scores: tensor([0.3131, 0.3121, 0.3096])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([52, 85, 88])
Top 3 document similarity scores: tensor([0.2186, 0.2167, 0.2049])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 95, 97])
Top 3 document similarity scores: tensor([0.2235, 0.2001, 0.1961])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([40, 52, 87])
Top 3 document similarity scores: tensor([0.1862, 0.1851, 0.1846])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([87, 88, 24])
Top 3 document similarity scores: tensor([0.4045, 0.4045, 0.1919])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([87, 88, 94])
Top 3 document similarity scores: tensor([0.4115, 0.4115, 0.3242])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([88, 87, 94])
Top 3 document similarity scores: tensor([0.3498, 0.3498, 0.2665])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([87, 88, 99])
Top 3 document similarity scores: tensor([0.4252, 0.4252, 0.1900])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([81, 87, 88])

```

```

Top 3 document similarity scores: tensor([0.1875, 0.1287, 0.1287])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([89, 88, 87])
Top 3 document similarity scores: tensor([0.4139, 0.3317, 0.3317])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([89, 99, 98])
Top 3 document similarity scores: tensor([0.4764, 0.2556, 0.2556])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([89, 49, 48])
Top 3 document similarity scores: tensor([0.3446, 0.3046, 0.3046])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([89, 88, 87])
Top 3 document similarity scores: tensor([0.5421, 0.2670, 0.2670])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([89, 88, 87])
Top 3 document similarity scores: tensor([0.6802, 0.3259, 0.3259])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([90, 45, 88])
Top 3 document similarity scores: tensor([0.3875, 0.2849, 0.2341])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([90, 74, 12])
Top 3 document similarity scores: tensor([0.4572, 0.1880, 0.1868])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([90, 92, 87])
Top 3 document similarity scores: tensor([0.4954, 0.3977, 0.3423])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([91, 84, 92])
Top 3 document similarity scores: tensor([0.3888, 0.3423, 0.3053])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([91, 89, 84])
Top 3 document similarity scores: tensor([0.5767, 0.2767, 0.2496])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([47, 46, 48])
Top 3 document similarity scores: tensor([0.3094, 0.3094, 0.2833])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([91, 82, 84])
Top 3 document similarity scores: tensor([0.4787, 0.2582, 0.2222])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([91, 82, 84])
Top 3 document similarity scores: tensor([0.5373, 0.2820, 0.2440])

```

```
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([17, 16, 13])
Top 3 document similarity scores: tensor([0.1699, 0.1332, 0.1271])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([91, 89, 84])
Top 3 document similarity scores: tensor([0.5674, 0.3214, 0.2957])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([91, 84, 89])
Top 3 document similarity scores: tensor([0.6039, 0.3524, 0.3221])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([97, 85, 88])
Top 3 document similarity scores: tensor([0.1702, 0.1372, 0.1281])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([58, 64, 81])
Top 3 document similarity scores: tensor([0.2655, 0.2263, 0.2215])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 92, 97])
Top 3 document similarity scores: tensor([0.4744, 0.4531, 0.4302])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([92, 48, 49])
Top 3 document similarity scores: tensor([0.5671, 0.3187, 0.3187])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([41, 58, 75])
Top 3 document similarity scores: tensor([0.2086, 0.1845, 0.1751])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([92, 90, 86])
Top 3 document similarity scores: tensor([0.3450, 0.2732, 0.2699])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([92, 37, 64])
Top 3 document similarity scores: tensor([0.4142, 0.2339, 0.2324])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([93, 84, 87])
Top 3 document similarity scores: tensor([0.6533, 0.3870, 0.3823])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 89, 93])
Top 3 document similarity scores: tensor([0.4348, 0.4092, 0.3534])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([90, 18, 93])
Top 3 document similarity scores: tensor([0.3473, 0.3425, 0.3344])
query_embedding shape: (384,)
```

```
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([93, 97, 92])
Top 3 document similarity scores: tensor([0.5049, 0.3593, 0.3135])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 89, 97])
Top 3 document similarity scores: tensor([0.3694, 0.3669, 0.3320])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([88, 87, 86])
Top 3 document similarity scores: tensor([0.2789, 0.2789, 0.2208])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([93, 88, 87])
Top 3 document similarity scores: tensor([0.3407, 0.2528, 0.2528])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([18, 86, 93])
Top 3 document similarity scores: tensor([0.2808, 0.2107, 0.2084])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([97, 93, 85])
Top 3 document similarity scores: tensor([0.3886, 0.3843, 0.3250])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([93, 97, 92])
Top 3 document similarity scores: tensor([0.5184, 0.4137, 0.3542])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([93, 88, 87])
Top 3 document similarity scores: tensor([0.4955, 0.3963, 0.3963])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([93, 97, 87])
Top 3 document similarity scores: tensor([0.5518, 0.3722, 0.3631])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 92, 82])
Top 3 document similarity scores: tensor([0.4252, 0.4197, 0.4038])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 93, 82])
Top 3 document similarity scores: tensor([0.4943, 0.4234, 0.4054])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([93, 97, 90])
Top 3 document similarity scores: tensor([0.6022, 0.4000, 0.3450])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([23, 69, 64])
Top 3 document similarity scores: tensor([0.2493, 0.2085, 0.1408])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
```

```
Top 3 document ids: tensor([93, 90, 89])
Top 3 document similarity scores: tensor([0.2879, 0.2741, 0.2684])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([93, 94, 88])
Top 3 document similarity scores: tensor([0.3700, 0.3277, 0.3072])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([94, 90, 84])
Top 3 document similarity scores: tensor([0.5024, 0.1631, 0.1611])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([94, 97, 84])
Top 3 document similarity scores: tensor([0.5707, 0.2660, 0.2026])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([49, 48, 50])
Top 3 document similarity scores: tensor([0.4050, 0.4050, 0.2978])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([94, 84, 18])
Top 3 document similarity scores: tensor([0.3711, 0.1683, 0.1653])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([94, 96, 42])
Top 3 document similarity scores: tensor([0.4813, 0.2622, 0.2232])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([94, 96, 50])
Top 3 document similarity scores: tensor([0.6377, 0.2399, 0.2234])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([94, 96, 50])
Top 3 document similarity scores: tensor([0.5080, 0.2295, 0.2201])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([94, 73, 72])
Top 3 document similarity scores: tensor([0.2496, 0.1937, 0.1895])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([94, 88, 87])
Top 3 document similarity scores: tensor([0.5804, 0.2124, 0.2124])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([95, 85, 93])
Top 3 document similarity scores: tensor([0.5343, 0.4523, 0.3861])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([95, 85, 92])
Top 3 document similarity scores: tensor([0.6144, 0.4125, 0.2886])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([95, 85, 93])
```

```
Top 3 document similarity scores: tensor([0.6914, 0.4813, 0.4304])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([64, 65, 40])
Top 3 document similarity scores: tensor([0.2252, 0.2132, 0.1768])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([95, 85, 93])
Top 3 document similarity scores: tensor([0.4980, 0.4414, 0.4045])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([95, 85, 86])
Top 3 document similarity scores: tensor([0.6139, 0.4286, 0.3337])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([95, 85, 93])
Top 3 document similarity scores: tensor([0.4066, 0.3492, 0.2677])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([64, 95, 65])
Top 3 document similarity scores: tensor([0.2208, 0.2031, 0.2023])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([95, 85, 93])
Top 3 document similarity scores: tensor([0.4204, 0.4085, 0.2942])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([96, 54, 50])
Top 3 document similarity scores: tensor([0.6783, 0.3036, 0.2614])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([96, 56, 52])
Top 3 document similarity scores: tensor([0.5897, 0.3428, 0.2804])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([32, 52, 2])
Top 3 document similarity scores: tensor([0.3190, 0.2858, 0.2712])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([96, 54, 52])
Top 3 document similarity scores: tensor([0.6353, 0.4175, 0.3105])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([58, 48, 49])
Top 3 document similarity scores: tensor([0.2322, 0.1904, 0.1904])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([37, 85, 92])
Top 3 document similarity scores: tensor([0.1757, 0.1711, 0.1681])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([63, 80, 4])
Top 3 document similarity scores: tensor([0.2442, 0.2359, 0.2243])
```



```
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([97, 93, 90])
Top 3 document similarity scores: tensor([0.4870, 0.3447, 0.3109])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 93, 97])
Top 3 document similarity scores: tensor([0.4843, 0.4515, 0.4012])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 92, 97])
Top 3 document similarity scores: tensor([0.4744, 0.4531, 0.4302])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([97, 93, 85])
Top 3 document similarity scores: tensor([0.3064, 0.2917, 0.2852])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 93, 97])
Top 3 document similarity scores: tensor([0.4928, 0.4612, 0.4357])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([92, 87, 88])
Top 3 document similarity scores: tensor([0.2120, 0.1997, 0.1997])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([99, 98, 92])
Top 3 document similarity scores: tensor([0.4302, 0.4302, 0.3223])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([85, 86, 95])
Top 3 document similarity scores: tensor([0.3641, 0.3317, 0.3271])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([95, 85, 99])
Top 3 document similarity scores: tensor([0.3729, 0.3709, 0.3608])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([84, 89, 82])
Top 3 document similarity scores: tensor([0.3676, 0.3557, 0.3348])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([99, 98, 84])
Top 3 document similarity scores: tensor([0.6665, 0.6665, 0.2992])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([99, 98, 86])
Top 3 document similarity scores: tensor([0.1924, 0.1924, 0.1869])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([99, 98, 87])
Top 3 document similarity scores: tensor([0.6231, 0.6231, 0.3109])
query_embedding shape: (384,)
```

```

similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([99, 98, 18])
Top 3 document similarity scores: tensor([0.3871, 0.3871, 0.2522])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([99, 98, 16])
Top 3 document similarity scores: tensor([0.3123, 0.3123, 0.1472])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([99, 98, 84])
Top 3 document similarity scores: tensor([0.6897, 0.6897, 0.3762])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([99, 98, 87])
Top 3 document similarity scores: tensor([0.3488, 0.3488, 0.2226])
query_embedding shape: (384,)
similarity_scores shape: torch.Size([100, 1])
Top 3 document ids: tensor([99, 98, 84])
Top 3 document similarity scores: tensor([0.5687, 0.5687, 0.2747])

```

Now evaluate the three versions of the sentence-based retriever. We only need to load it once.

```

sentence_retriever = SentenceEmbeddingRetriever(
    documents=retrieval_dataset.documents,
    model_name='multi-qa-MiniLM-L6-cos-v1'
)

```

Initialization: embedding documents...

100%

100/100 [00:03<00:00, 40.84it/s]

Exercise: Fill in the missing lines in the cell below.

```

for summary_statistic in ['mean', 'min', 'max']:

    all_similarity_scores = []

    for query in retrieval_dataset.queries:
        _, _, similarity_scores = sentence_retriever.retrieve(
            query=query,
            mode=summary_statistic
        )
        all_similarity_scores.append(similarity_scores)

    # Convert the list of tensors into a single tensor
    all_similarity_scores = torch.stack(all_similarity_scores)
    print(f"\nShape of all_similarity_scores for mode '{summary_statistic}'")
    print(f"First 5x5 slice of all_similarity_scores for mode '{summary_s

    # Now compute accuracy for each value of k

```

```

for k in [1, 3, 5, 10]:
    acc = top_k_accuracy_score(
        retrieval_dataset.true_indices, # these are the target documents
        all_similarity_scores,
        k=k
    )

    # Store the top-k accuracy score in the top_k_accuracy data structure
    top_k_accuracy[k][f"sentence_{summary_statistic}"] = acc

```

```

Shape of all_similarity_scores for mode 'mean': torch.Size([632, 100])
First 5x5 slice of all_similarity_scores for mode 'mean':
tensor([[ 0.1870,  0.0653,  0.0514,  0.0095,  0.1029],
        [ 0.3226,  0.0482, -0.0420, -0.0255,  0.0736],
        [ 0.2762,  0.1079,  0.0386,  0.0231,  0.0632],
        [ 0.2460,  0.0672,  0.0263,  0.0097,  0.0037],
        [ 0.2688,  0.1084,  0.0354,  0.0151,  0.1074]])

Shape of all_similarity_scores for mode 'min': torch.Size([632, 100])
First 5x5 slice of all_similarity_scores for mode 'min':
tensor([[ -0.0965, -0.0394, -0.0938, -0.1362, -0.1407],
        [  0.0170, -0.0764, -0.2473, -0.1864, -0.1576],
        [ -0.0614, -0.0454, -0.1056, -0.0966, -0.0319],
        [ -0.0588, -0.0098, -0.1364, -0.1190, -0.1572],
        [ -0.0185, -0.0256, -0.1458, -0.1022, -0.0402]])

Shape of all_similarity_scores for mode 'max': torch.Size([632, 100])
First 5x5 slice of all_similarity_scores for mode 'max':
tensor([[ 0.4605,  0.2724,  0.4871,  0.3108,  0.3591],
        [ 0.6331,  0.2149,  0.1746,  0.1183,  0.2870],
        [ 0.7301,  0.2459,  0.2346,  0.2132,  0.2200],
        [ 0.5997,  0.1679,  0.3403,  0.2735,  0.2019],
        [ 0.6842,  0.2508,  0.2548,  0.1329,  0.2614]])

```

Let's see how the models fair!

```

top_k_accuracy_df = pd.DataFrame(top_k_accuracy)
top_k_accuracy_df

```

	1	3	5	10	
paragraph	0.582278	0.754747	0.805380	0.857595	
sentence_mean	0.518987	0.707278	0.778481	0.865506	
sentence_min	0.155063	0.305380	0.376582	0.545886	
sentence_max	0.661392	0.806962	0.846519	0.906646	

Next steps:
 [Generate code with top_k_accuracy_df](#)
[New interactive sheet](#)

2025-10-18: ALR - sentence level encoding with max stistic works the mest

[Optional] In the experiments above, we picked a language model and kept it fixed. However, the language model is itself an experimental variable—and a very important one, as the choice of language model determines the quality of the embeddings.

Compare the retrieval systems above against systems based on two alternative language models: [all-MiniLM-L6-v2](#) and [msmarco-distilbert-base-v4](#). For the sentence-based model, you don't need to evaluate all three versions. Pick only the one with the summary statistic that performed best.

[Optional] *Write code to reproduce the experiments above using alternative language models.*

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

[Optional] In the experiments above, we used news stories as documents and their respective questions as queries. However, it is reasonable to think that summaries might be better proxies of users' queries in some real-world scenarios.

For convenience, we provide a Dataset class analogous to the RetrievalDataset class above

```
# RetrievalDatasetWithSummaries class
# Please execute this cell.

class RetrievalDatasetWithSummaries:

    def __init__(self, dataset, num_documents=None):
        self.dataset = dataset

        if num_documents:
            self.num_documents = num_documents
            self.documents = self.dataset['train']['story'][:self.num_documents]
            self.queries = self.dataset['train']['summary'][:self.num_documents]
            self.num_queries = len(self.queries)
        else:
            self.documents = self.dataset['train']['story']
            self.queries = self.dataset['train']['summary']
            self.num_documents = len(self.documents)
            self.num_queries = len(self.queries)

        self.true_indices = []
        for i, qs in enumerate(self.dataset['train']['summary'][:self.num_d
```

```
for _ in qs:
    self.true_indices.append(i)
```

```
retrieval_dataset = RetrievalDatasetWithSummaries(dataset, num_document
```

[Optional] *Repeat the experiments above using summaries as queries.*

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

✓ Summarization

The second module is an automatic summarizer: it takes as input a news story and it generates a short summary.

We will test two types of summarizers:

1. an instruction fine-tuned (multi-task) language model: this is a language model first pre-trained on the next-word prediction task, then fine-tuned on a dataset of instruction-execution pairs for a large variety of tasks
2. a language model fine-tuned specifically for summarization: this is a language model first pre-trained on the next-word prediction task, then fine-tuned on text-summary pairs.

✓ Instruction fine-tuned summarizer

As a multi-task instruction fine-tuned language model, we will use [google/flan-t5-small](#). If there are enough computational resources available, you can use a larger model, e.g. [google/flan-t5-large](#)

This model has been trained on 1.8K tasks, including summarization, phrased as instructions.

```
model_name_t5 = "google-t5/t5-large" #replace with "google-t5/t5-large"
tokenizer_t5 = T5Tokenizer.from_pretrained(model_name_t5)
model_t5 = T5ForConditionalGeneration.from_pretrained(model_name_t5, de
```

```
spiece.model: 100% 792k/792k [00:00<00:00, 57.9MB/s]

tokenizer.json: 1.39M/? [00:00<00:00, 49.3MB/s]

config.json: 1.21k/? [00:00<00:00, 91.5kB/s]

model.safetensors: 100% 2.95G/2.95G [05:00<00:00, 13.4MB/s]
```

Let's test it at work on a news story!

```
test_news_story = dataset["train"][3]["story"]
```

As this is an instruction fine-tuned model, we can simply kindly ask it to summarize the story in plain words:

```
print('Instruction:\n')
input_text = f"Summarize: {test_news_story}"
pprint(input_text)
```

Instruction:

Summarize: (AOL Autos) -- Collecting cars is an expensive hobby. Here's a fortune as a collectible years from now. The Audi S5 is a slick coupe need to break the bank to buy a hot and value-appreciating ride. But you "Many consumers could be driving a future collector car right now," McKe of collector cars in the United States, and himself a respected collecto cars that are currently on the road but may be considered nostalgic in 1 possesses a 'buzz,' a wow factor that resonates with consumers of all ag heads already." Why do some vehicles become tomorrow's collector car? Ha popularity, limited production numbers and the style of the next generat 10 collector cars of the future: 1. Cadillac XLR-V Roadster Under the ho supercharged and puts out 443 horsepower. This is a serious domestic roa Porsche 911 Cabriolet and Jaguar XKR, as well as the four-seat BMW M6 co Lotus Exige S Most practical people will find the Lotus Exige is a misera for a grocery getter! Research uncovered one anonymous review stating "B think about this car ... it is that rewarding." AOL Autos: Lotus Exige S credentials. It provides good looks, impressive all-wheel-drive handling AOL Autos: Audi S5 4. Mustang Shelby GT 500 KR The new GT500 KR (King of turns it into a 540 horsepower muscle car that pays tribute to the legen be around 1,000. AOL Autos: Mustang Shelby GT 500 KR 5. Chevrolet Corvet with a 7.0-liter V8 that produces 505 horsepower. The last 50 years have collectible. AOL Autos: Chevrolet Corvette Z06 6. Smart At first look, th not. Rather, it is a marvel of packaging efficiency. This is the first year United States. 7. Subaru Impreza WRX STi Some Subaru enthusiasts may not but this will make it more collectible down the road. 8. Honda S2000 CR built. This is a high-performance version of the stock S2000 that is reg Pontiac Solstice/Saturn Sky This pair of roadsters from GM have all the powerful engine, independent suspension and an impressive weight balance HEMI V8 with 425 horsepower and 420 ft.-lbs. of torque. Need we say more

```
# better prompting
```

```
def build_t5_prompt(article, style="concise", audience="general"):
    return (
        "summarize: "
        f"Write a {style} summary for a {audience} audience. "
        "Keep key facts, people, dates, numbers, and avoid speculation. "
        "Return 5-8 sentences.\n\n"
        "Article:\n"
        f"{article}"
    )
input_text = build_t5_prompt(test_news_story)
```

Now we tokenize the instruction text.

```
device = "cuda" if torch.cuda.is_available() else "cpu"

# Tokenize input
input_ids = tokenizer_t5(input_text, return_tensors="pt", max_length=51
```

Then, we generate a summary. We set the minimum and maximum length of the summary to 30 and 100 tokens, respectively.

Exercise: Experiment with different minimum and maximum length values by changing the parameters of the `generate` function below.

```
# Generate summary conditioned on instruction and original news story
outputs = model_t5.generate(
    input_ids,
    min_new_tokens=128,    # minimum summary length #note (T5-large is ~
    max_new_tokens=512,    # maximum summary length
    do_sample=True        # so we obtain the same sample every time
)
print("Summary:\n")
pprint(tokenizer_t5.decode(outputs[0], skip_special_tokens=True))
```

Summary:

```
cnn's robert patrick highlights the best new cars on the market . in 15
considered nostalgic . a new audi offers good value, but it could go on t
serious sports car -- makes the list . a 2006 bmw 328i sports car has a
cadillac srx- a " n "-a .&
```

Output for small model and 30/10 min/max tokens: "Summary:

collector cars. a new car today:... a collection car today. collector cars. Here's a twist:
Buy a new car today that will be worth a fortune as a collectible years from now...: 1.
Cadillac XLR-V Roadster Under the hood sits a hand-built 4.4-liter V8 that's
supercharged and puts out"

Output with large model and 30/100 min/max tokens: "Summary:

a slick coupe with solid credentials.R is a slick coupe with solid credentials.R is a hot car.R is a hot car. AOL Autos: Shelby GT500 KR 5. Audi S5R is a hot car. is an expensive hobby.. is S5. " or even XLR XLR 3..."

Output large model and 300/1000 min/max tokens: "Summary:

a slick coupe with solid credentials.R is a slick coupe with solid credentials.R is a hot car.R is a hot car. AOL Autos: Shelby GT500 KR 5. Audi S5R is a hot car. is an expensive hobby.. is S5. " or even XLR XLR 3..... 5.RR a a a a- a- a a a " the-S is enjam. 5. Shelby - - - - -

• ○ ■ -"

✓ Putting it all together

Now that we have tested and understood the components of the summarizer, we can compile them in a Python class.

Fill in the minimum and maximum length based on your experiments in the cell above.

```
class InstructionFinetunedSummarizer:

    # Initialize model and tokenizer
    def __init__(
        self,
        model_name="google/flan-t5-large",
        device=None
    ):
        self.model = T5ForConditionalGeneration.from_pretrained(model_name)
        self.tokenizer = T5Tokenizer.from_pretrained(model_name)
        self.device = device

    # Summarizing function
    def summarize(self, input_texts):

        # Tokenize texts
        summariser_inputs = []
        for input_text in input_texts:
            summariser_inputs.append(
                self.tokenizer(input_text, return_tensors="pt", max_length=51
                               truncation=True).input_ids.to(self.device)
            )

        # Generate summaries
        outputs = []
        for input_ids in summariser_inputs:
```



```

        outputs.append(
            self.model.generate(
                input_ids,
                min_new_tokens=128,
                max_new_tokens=512,
                do_sample=False
            )[0]
        )

    # Transform token ids into actual words
    summaries = self.tokenizer.batch_decode(outputs, skip_special_token=True)

    return summaries

```

The summarizer is ready! We will evaluate it later, together with the fine-tuned summarizer below.

✓ Fine-tuned summarizer

As a fine-tuned summarizer, you will use [ainize/bart-base-cnn](#). This is a language model that has been finetuned specifically on summaries of CNN and DailyMail news stories. If you have enough computational resources, you can use the larger model:

[facebook/bart-large](#)

Let's load the model using the Hugging Face `pipeline` functionality, which makes the code more concise.

```

summarizer = pipeline("summarization", model="ainize/bart-base-cnn", to_device=-1)

```

config.json:	1.58k/? [00:00<00:00, 142kB/s]
vocab.json:	899k/? [00:00<00:00, 34.0MB/s]
merges.txt:	456k/? [00:00<00:00, 21.8MB/s]
tokenizer.json:	1.36M/? [00:00<00:00, 46.6MB/s]

Device set to use cuda:0

Generate a summary for our test news story.

```

test_summary = summarizer(test_news_story)[0]["summary_text"]

print('Summary:\n')
pprint(test_summary)

```

Summary:

Buy a new car today that will be worth a fortune as a collectible years solid credentials . Cadillac XLR-V Roadster under the hood sits a hand-b

For comparison - chat gpt answer:

AOL Autos highlights a unique approach to car collecting: buying new cars today that could become valuable collectibles in 15–20 years. According to McKeel Hagerty, CEO of Hagerty Insurance, future collector cars tend to combine cultural appeal, limited production, and distinctive design that resonates with the next generation.

Hagerty's "Hot List" of the top 10 future collectible cars includes:

Cadillac XLR-V Roadster – Supercharged 4.4L V8, 443 hp, luxury performance rivaling top European convertibles.

Lotus Exige S – A raw, thrilling sports car for purists.

Audi S5 – Stylish, high-performing coupe at a relatively af

✓ Putting it all together

The `pipeline` tool already compiles different components together, but let's define a Python class for this fine-tuned summarizer for extra comfort.

```
class FinetunedSummarizer:

    # Initialize model and tokenizer
    def __init__(
        self,
        model_name="facebook/bart-large-cnn",
        device=None
    ):
        self.summarizer = pipeline("summarization", model=model_name, token
        self.device = device
        if self.device is None:
            self.device = "cuda" if torch.cuda.is_available() else "cpu"

    # Summmarizing function
    # It takes texts as input and it returns summaries
    def summarize(self, input_texts):
        outputs = self.summarizer(input_texts)
        summaries = [output["summary_text"] for output in outputs]
        return summaries
```

✓ Evaluation

How good are the summaries produced by these two models? Is the fine-tuned model

better, as we would probably expect given its task-specific training data?

To assess and compare summarizers, you will use two standard natural language generation (NLG) metrics that are thought to produce good proxies of summary quality:

- **BERTScore**: BERTScore leverages contextualised word embeddings extracted from a pre-trained language model and matches words in candidate and reference sentences by cosine similarity. It has been shown to correlate with human judgment on sentence-level and system-level evaluation. BERTScore can be computed as precision, recall, and F1, which can be useful for evaluating different language generation tasks.
- **ROUGE**: Recall-Oriented Understudy for Gisting Evaluation (ROUGE) is a set of metrics used for evaluating automatic summarization and machine translation. You will use only ROUGE-L, which measures the longest common subsequence between a generated summary and a reference summary.

Both metrics are *reference-based*, i.e., they assess the quality of a summary in terms of its closeness to a reference summary, typically produced by a human. Here, you are going to use the summaries in the dataset as references.

We define convenience functions for the evaluation metrics.

```
# Evaluation metrics
# Please execute this cell.

import evaluate

bertscore = evaluate.load("bertscore")
rouge = evaluate.load('rouge')

def evaluate_summaries(predictions, references):

    # Compute BERTScore
    bertscores = bertscore.compute(
        predictions=predictions,
        references=references,
        lang="en"
    )

    # Take the average BERTScore over all summaries
    result_dict = {
        f'bertscore_{stat}': np.mean(bertscores[stat]) for stat in bertsc
    }

    # Compute ROUGE (average over all summaries taken automatically)
    rouge_scores = rouge.compute(
        predictions=predictions,
        references=references
    )
```

```
result_dict['rougeL'] = rouge_scores['rougeL']

return result_dict
```

Downloading builder script: 7.95k/? [00:00<00:00, 752kB/s]

Downloading builder script: 6.27k/? [00:00<00:00, 497kB/s]

Model comparison

Let's compare the instruction fine-tuned model against the summarization fine-tuned one using BERTScore and ROUGE.

Load models and data. We will only use a subset of the data to speed up evaluation.

Feel free to use a larger subset, but beware of the longer runtimes.

```
instruction_model = InstructionFinetunedSummarizer()
finetuned_model = FinetunedSummarizer()

dev_set_news_stories = dataset['train']['story'][:30]
dev_set_summaries = dataset['train']['summary'][:30]
```

```
config.json: 100% 662/662 [00:00<00:00, 40.7kB/s]
model.safetensors: 100% 3.13G/3.13G [02:23<00:00, 27.6MB/s]
generation_config.json: 100% 147/147 [00:00<00:00, 14.1kB/s]
tokenizer_config.json: 2.54k/? [00:00<00:00, 246kB/s]
spiece.model: 100% 792k/792k [00:00<00:00, 329kB/s]
special_tokens_map.json: 2.20k/? [00:00<00:00, 193kB/s]
tokenizer.json: 0.40M/? [00:00<00:00, 58.0MB/s]
```

Generate summaries for the first 5 news stories, to save time (you can increase this).

```
summaries_instruction = instruction_model.summarize(dev_set_news_stories[:5])
summaries_finetuned = finetuned_model.summarize(dev_set_news_stories[:5])
```

Print out the evaluation results.

```

instruction_results = evaluate_summaries(summaries_instruction, dev_set)
finetuned_results = evaluate_summaries(summaries_finetuned, dev_set_summa

print('\nInstruction Fine-Tuned Summarizer')
print(instruction_results)

print('\nFine-Tuned Summarizer')
print(finetuned_results)

```

```

tokenizer_config.json: 100%          25.0/25.0 [00:00<00:00, 2.67kB/s]

config.json: 100%                   482/482 [00:00<00:00, 58.9kB/s]

vocab.json: 100%                   899k/899k [00:00<00:00, 2.08MB/s]

merges.txt: 100%                   456k/456k [00:00<00:00, 2.08MB/s]

tokenizer.json: 100%               1.36M/1.36M [00:00<00:00, 2.04MB/s]

model.safetensors: 100%            1.42G/1.42G [02:43<00:00, 13.2MB/s]

```

Some weights of RobertaModel were not initialized from the model checkpo

✓ Evaluating the evaluation metrics

NLP metrics are far from perfect. They act as a proxy for human feedback when this is not available. We will now evaluate the evaluation metrics themselves by testing whether their produced scores correspond to human intuition.

Evaluate a few summaries produced by the fine-tuned model giving them a score from 1 to 4, where:

1. the summary doesn't match the content of the story
2. the summary partially matches to content of the story but it doesn't convey the story's main points
3. the summary partially matches the content of the story and it conveys the story's main points
4. the summary matches the content of the story and it conveys the story's main points

Here, we print 5 stories and their respective summaries generated by the summary fine-tuned model. Feel free to evaluate more than 5.

```
# feel free to evaluate more than 5 summaries, it will give you a better
N_manual = 5

for i in range(N_manual):
    pprint(f"STORY {i+1}:\n {dev_set_news_stories[i]}")
    print()
    pprint(f"SUMMARY {i+1}: {summaries_finetuned[i]}")
    print('\n-----\n')
```

STORY 1: 'SINDH KALAY', England (CNN) -- The aroma of freshly baking flatbread fills the air as British soldiers position themselves for a quick patrol around the villa in the mock Afghan village of Sindh Kalay. Market vendors hawk grapes and smoking water pipes and suspicious-looking men lurk beside battered motorcycles. Should we search? Approach the village elders first? In the complex political environment, what is the best course of action? Except this is not Afghanistan. It's Norfolk, England. It is the green landscape and tidy farmhouses of the English countryside. This is the British Army's state-of-the-art training ground. It cost more than \$20 million to build. Every soldier serving in Afghanistan will do his or her training here. "I think it's better than being in Afghanistan itself," says Col. David Colthup of the 2nd Battalion. Soldiers already served one tour of duty in Afghanistan's Helmand province and are now in Helmand province are tasked with mentoring and training Afghan security forces. Helmand province is a stronghold and Afghanistan's center of opium production. "Ultimately, a soldier's job is to win the war. That's what a soldier trains to do. But today, it's a much, much more complex job. It's the business of being able to interact either through an interpreter or through the police or army. And to understand how the people operate and how we can win the war, that's what it's about," he says. The most distinctive features of Sindh Kalay are the narrow alleyways up the village compound, creating narrow alleyways difficult for troops to move through. They are for asylum-seekers, many of whom have fled the Taliban. They play the roles of Afghan security forces. Several Afghan women are also on hand, useful for their knowledge of the cultural sensitivities of entering an Afghan home. Watch British troops as they train. Insurgents are played by Nepalese Gurkha soldiers authorized to handle the role. They are to partake in the Pashtun banter among the Afghans. Fazel Beria is also responsible for recruiting and for creating the sights and smells of Sindh Kalay. He is an Afghan in the market in Western clothes. He beams with pride walking down the street. He is training the British Army. "Everything with the culture comes up with the training. You want to win that, you need to know about their culture. You need to know their way of life." He gives high marks to the soldiers training so far. After a week, he tells the soldiers about what went wrong and what went right. Sometimes, it has been quite a lot of surprises," Beria says. Like Afghanistan, Sindh Kalay cannot do that," he laughs. "The Afghan will be sitting very comfortable in Sindh Kalay. He will get used to it." See photos of British troops on patrol in Sindh Kalay. The village the army trained on farmhouses and in urban neighborhoods that resembled Sindh Kalay does more than mimic the physical reality of Afghanistan. It is a training ground. Troops are grilled in

SUMMARY 1: Sindh Kalay is the British Army's state-of-the-art training ground and every British soldier will do training here. The village is staffed with Afghan women who have fled the Taliban. Taliban insurgents are played by Nepalese Gurkha soldiers.

STORY 2: (AOL Autos) -- At the 2009 Detroit Auto Show, Chrysler, Mercedes-Benz and other manufacturers unveiled electric vehicles of the future will look like. And the future of driving will be faster. 0-to-60 mph in around 4 seconds. Those in the know realize that sometime in the future, cars and trucks in the US will feature electric final drive systems. The motor will be replaced by batteries, fuel cells, on-board generators, and perhaps even the sun. But the reality of this reality. Given our current economic times, reality

ideas of what electric vehicles (or "EVs" for short) might actually look like. The electric vehicles Chrysler showed in Detroit, Michigan, were shown at the Detroit Auto Show. Each of these vehicles is a running prototype, not some pie-in-the-sky-wish. A group of engineers at Chrysler that develops the company's EVs. To date, Chrysler has three powered models, each quite different from the other: a Dodge Circuit EV is a minivan (front-wheel-drive), a Jeep Wrangler Unlimited (all-wheel-drive). Chrysler promises to offer at least one of these models in 2010. The Circuit EV photos Chrysler approaches electric vehicles with simple plug-in hybrids. These vehicles use similar electric drive motors (only varying in power output and battery management controller). Each plugs in to 110- or 220-volt household outlets.

Exercise: Provide your scores for the summaries in order in the list below.

```
your_scores = [
    3, # missing: recreating culture
    2, # missing feature, innovation
    4, # ok
    3, # missing why they will be expensive (pop culture, rarity, style)
    4 # ok
]
```

Now let's see what automatic evaluation scores they get. Do they match your judgements?

```
bertscores = bertscore.compute(
    predictions=summaries_finetuned[:N_manual],
    references=dev_set_summaries[:N_manual],
    lang="en"
)

rouge_scores = rouge.compute(
    predictions=summaries_finetuned[:N_manual],
    references=dev_set_summaries[:N_manual],
    use_aggregator=False
)

print('BertScore:')
for score_name, score in bertscores.items():
    print(f' {score_name}: {score}')

print()
print('Rouge:')
print(f' {rouge_scores["rougeL"]}')

```

```
BertScore:
precision: [0.8609325885772705, 0.8849539756774902, 0.886622428894043,
recall: [0.8654041290283203, 0.8620781898498535, 0.8909502029418945,
f1: [0.8631625771522522, 0.8733662962913513, 0.8887810111045837, 0.906
hashcode: roberta-large_L17_no-idf_version=0.3.12(hug_trans=4.57.1)

Rouge:
[0.29473684210526313, 0.27450980392156865, 0.396039603960396, 0.603773
```

This is exactly how evaluation metrics are evaluated in NLP research. Human quality judgements are collected and then their correlation with the evaluation metric scores is computed. Here, we do not compute correlations as the number of scores to compare is too low.

✓ [OPTIONAL] Question Answering

The third module is a Question Answering system: it takes as input a news story and a question and it generates a short answer.

You will test two types of QA systems:

1. an instruction fine-tuned (multi-task) language model
2. a language model fine-tuned specifically for question answering

✓ Instruction Fine-Tuned

As a multi-task instruction fine-tuned language model, you will again use [google/flan-t5-small](https://huggingface.co/google/flan-t5-small). Question answering is among the 1.8K tasks this model was trained on.

Start coding or [generate](#) with AI.

```
model_name_t5 = "google/flan-t5-small" # replace with "google/flan-t5-lar
tokenizer_t5 = T5Tokenizer.from_pretrained(model_name_t5)
model_t5 = T5ForConditionalGeneration.from_pretrained(model_name_t5, de
```

Let's see it at work on a sample question!

```
test_news_story = dataset["train"][1]["story"]
```

```
question = "What will power the motors used in electric cars and trucks
```

Again, as this is an instruction fine-tuned model, we can simply provide it with a news story and ask it a question.

```
input_text = f"Context: {test_news_story}\n Question: {question}" # \n
pprint(input_text)
```


Context: (AOL Autos) -- At the 2009 Detroit Auto Show, Chrysler, Merced electric vehicles of the future will look like. And the future of drivin 0-to-60 mph in around 4 seconds. Those in the know realize that sometime and trucks in the US will feature electric final drive systems. The motor batteries, fuel cells, on-board generators, and perhaps even the sun. Bu inevitability of this reality. Given our current economic times, reality ideas of what electric vehicles (or "EVs" for short) might actually look electric vehicles Chrysler showed in Detroit, Michigan, were shown at ot Each of these vehicles is a running prototype, not some pie-in-the-sky-w group of engineers at Chrysler that develops the company's EVs. To date, powered models, each quite different from the other: a Dodge Circuit EV s Country minivan (front-wheel-drive), a Jeep Wrangler Unlimited (all-whee drive). Chrysler promises to offer at least one of these models in 2010, Circuit EV photos Chrysler approaches electric vehicles with simple plug vehicles uses similar electric drive motors (only varying in power outpu management controller. Each plugs in to 110- or 220-volt household outle use an on-board range-extending battery charger (a generator). This gene initial batter charge has been spent (usually within a range of 40 miles vehicles an estimated range of approximately 400 miles. The generator is runs with exceptional efficiency. This technology is similar in concept to Chevrolet Volt, a vehicle that should be ready for production in 2010. A Circuit carries a larger battery pack and no generator, so its range on miles. Its large battery pack combined with compact dimensions and the e blast the car from zero-to-sixty mph in around four seconds, exceptional type. Mercedes-Benz Mercedes-Benz used the 2009 Detroit Auto Show to sho Mercedes approach was to develop one efficient body style, and then equip packages. AOL Autos: Mercedes Stirling Moss photos Much of the hardware propulsion units is built into what Mercedes calls "sandwich-floor" archi production cars. The design helps keep heavy components mounted low on t and maximized interior room. All three Concept BlueZERO vehicles include large battery pack that is said to deliver a range of 120 miles. The F-C supplements the vehicle's range with a hydrogen fuel cell. The fuel cell pack that extends cruising range to 240 miles. The E-Cell Plus, with a r champion. The key is the on-board generator powered by tiny 1-liter turb Question: What will power the motors used in electric cars and trucks?

Now let's tokenize the instruction text and pass it to the model for conditional generation.

We set the minimum and maximum length of the generated answer to 5 and 30 tokens, respectively. ***Experiment with different minimum and maximum length values by changing the parameters of the `generate` function below.***

```
# Tokenize input
device = "cuda" if torch.cuda.is_available() else "cpu"
input_ids = tokenizer_t5(input_text, max_length=512, truncation=True, ret

# Generate summary conditioned on instruction and original news story
outputs = model_t5.generate(
    input_ids,
    min_new_tokens=64,      # minimum answer length
    max_new_tokens=512,    # maximum summary length
    do_sample=False        # so we obtain the same sample every time
)
```

```
pprint(tokenizer_t5.decode(outputs[0], skip_special_tokens=True))
```

Chrysler's EVs are powered by a small gasoline-powered engine that runs both Jeeps use an on-board range-extending battery charger (a generator) batter charge has been spent (usually within a range of 40 miles), suppl vehicles an estimated range of approximately 400 miles.

When switched from "google/flan-t5-small" to "google/flan-t5-large" quality deteriorated

Small model 32/512 min/max tokens:

Chrysler's EVs are powered by a small gasoline-powered engine that runs with exceptional efficiency. The Chrysler and both Jeeps use an on-board range-extending battery charger (a generator)

Small model 64/512 min/max tokens:

Chrysler's EVs are powered by a small gasoline-powered engine that runs with exceptional efficiency. The Chrysler and both Jeeps use an on-board range-extending battery charger (a generator) that turns on after the vehicle's initial batter charge has been spent (usually within a range of 40 miles), supplying extra voltage that give these three vehicles an estimated range of approximately 400 miles.

Large model 32/512 min/max tokens:

Chrysler's EVs are a step in the right direction. They're a step in the right direction for the future of driving.

✓ Putting it all together

Now that we have tested and understood the components of the QA system, we can compile them in a Python class.

Exercise: Fill in the minimum and maximum length based on your experiments in the cell above.

```
class InstructionFinetunedQA:

    # Initialize model and tokenizer
    def __init__(
        self,
        model_name="google/flan-t5-small", # changed from "google/flan-t5-lar
        device=None
    ):
        self.device = device if device is not None else ("cuda" if torch.cu
        self.model = T5ForConditionalGeneration.from_pretrained(model_name)
        self.tokenizer = T5Tokenizer.from_pretrained(model_name)
```

```

# Question answering function
# It takes as input multiple texts and multiple questions
# (one per text) and it returns the corresponding answers
def qa(self, input_texts, questions):
    assert len(input_texts) == len(questions)

    # Tokenize input text: news stories and questions
    qa_inputs = []
    for text, question in zip(input_texts, questions):
        qa_inputs.append(
            self.tokenizer(
                f"Context: {text}\n Question: {question}",
                return_tensors="pt"
            ).input_ids.to(self.device)
        )

    # Generate answers
    outputs = []
    for qa_input in qa_inputs:
        outputs.append(
            self.model.generate(
                qa_input,
                min_new_tokens=64,
                max_new_tokens=512,
                do_sample=False
            )[0]
        )

    # Transform token ids into actual words
    answers = self.tokenizer.batch_decode(outputs, skip_special_tokens=

    return answers

```

✓ Fine-tuned QA

As a fine-tuned summarizer, you will use [deepset/roberta-base-squad2](#). This is a language model fine-tuned on question-answer pairs.

```

model_name_qa = "deepset/roberta-base-squad2"
qa_pipeline = pipeline('question-answering', model=model_name_qa, token

config.json: 100% 571/571 [00:00<00:00, 61.2kB/
s]

model.safetensors: 100% 496M/496M [00:13<00:00, 69.7MB/
s]

tokenizer_config.json: 100% 79.0/79.0 [00:00<00:00, 1.33kB/
s]

vocab.json: 899k/2 [00:00<00:00, 12.0MB/s]

```

vocab.json. 0.000 : [00.00<00.00, 12.0MB/s]

Let's test it on the same question as above.

```
test_news_story = dataset["train"][1]["story"]
question = "What will power the motors used in electric cars and trucks"
```

Here, we need to provide text and question in a dictionary but they are used as a single string of text under the hood, just like before. After all, this is a language model too.

```
qa_input = {
    'question': question,
    'context': test_news_story
}

qa_output = qa_pipeline(qa_input)

pprint(qa_output['answer'])

batteries, fuel cells, on-board generators, and perhaps even the sun
/usr/local/lib/python3.12/dist-packages/transformers/pipelines/question_
warnings.warn(
```

✓ Putting it all together

Let's compile the fine-tuned QA system into a Python class for ease of use.

```
class FinetunedQA:

    # Initialize model and tokenizer
    def __init__(
        self,
        model_name="deepset/roberta-base-squad2",
        device=None
    ):
        self.qa_pipeline = pipeline('question-answering', model=model_name,
        self.device = device
        if self.device is None:
            self.device = "cuda" if torch.cuda.is_available() else "cpu"

    # Question answering function
    # It takes as input multiple texts and multiple questions
    # (one per text) and it returns the corresponding answers
    def qa(self, input_texts, questions):
        assert len(input_texts) == len(questions)

        # Prepare input: texts + questions
        qa_inputs = []
        for text, question in zip(input_texts, questions):
```

```

        qa_inputs.append({
            'context': text,
            'question': question
        })

    # Generate answers
    outputs = self.qa_pipeline(qa_inputs)

    # Deal with the special case where only one
    # text-question pair is provided as input
    if type(outputs) is dict:
        answers = outputs["answer"]
    else:
        answers = [output["answer"] for output in outputs]

    return answers

```

✓ Evaluation

How good are the questions produced by these two models? Is the fine-tuned model better?

To assess and compare QA systems, you will use two reference-based NLG evaluation metrics:

- **BERTScore**: Remember, BERTScore leverages contextualised word embeddings extracted from a pre-trained language model and matches words in candidate and reference sentences by cosine similarity. It is a general way to measure the similarity between two pieces of text, so it be useful for evaluating different language generation tasks.
- **Exact Match**: Because answers are typically short, it is reasonable to expect them to exactly match the human reference. This is what Exact Match does: it is 1 when the generated answer matches the reference, and 0 otherwise. We will report the relative amount of exact matches across all question-answer pairs.

✓ Evaluation metrics

```

# @title Evaluation metrics
# @markdown Please execute this cell

bertscore = evaluate.load("bertscore")
exact_match = evaluate.load("exact-match")

def evaluate_qa(predictions, references):

    # Compute BERTScore
    bertscores = bertscore.compute(
        predictions=predictions,
        references=references
    )

```

Please execute this cell.



```

        references=references,
        lang="en"
    )

    # Take the average BERTScore over all
    result_dict = {
        f'bertscore_{stat}': np.mean(scores, axis=0)
    }

    # Compute percentage of exact matches
    exact_match_scores = exact_matcher.compute_exact_match_scores(
        predictions=predictions,
        references=references,
        lang="en"
    )

    result_dict['exact_match'] = exact_match_scores

    return result_dict

```

Downloading builder script: 5.67k/? [00:00<00:00, 346kB/s]

✓ Model comparison

Let's compare the instruction fine-tuned model against the fine-tuned one.

Load the models.

```

instruction_model = InstructionFinetunedQA()
finetuned_model = FinetunedQA()

```

Device set to use cuda:0

Load the data. Multiple question-answer pairs are available for each story. For simplicity, you'll only use one.

```

dev_set_news_stories = dataset['train']['story']

# For each story, take the first question and first answer only
dev_set_questions = [dataset['train'][i]['questions'][0] for i in range(len(dev_set_news_stories))]
dev_set_answers = [dataset['train'][i]['answers'][0][0] for i in range(len(dev_set_news_stories))]

```

We'll limit the data to a subset of the original dataset consisting of 30 data points. *Feel free to use a larger subset, but beware of the longer runtimes.*

```

dev_set_news_stories = dev_set_news_stories[:30]
dev_set_questions = dev_set_questions[:30]
dev_set_answers = dev_set_answers[:30]

```

Conditionally generate answers.

```
answers_instruction = instruction_model.qa(dev_set_news_stories, dev_se
answers_finetuned = finetuned_model.qa(dev_set_news_stories, dev_set_ques
```

Token indices sequence length is longer than the specified maximum sequen

Print out the evaluation results.

```
print('Instruction Fine-Tuned QA')
print(evaluate_qa(answers_instruction, dev_set_answers))
```

```
print()
```

```
print('Fine-Tuned QA')
print(evaluate_qa(answers_finetuned, dev_set_answers))
```

```
Instruction Fine-Tuned QA
Some weights of RobertaModel were not initialized from the model checkpo
You should probably TRAIN this model on a down-stream task to be able to
{'bertscore_precision': np.float64(0.7617560267448426), 'bertscore_recall

Fine-Tuned QA
{'bertscore_precision': np.float64(0.9039163708686828), 'bertscore_recall
```

✓ Evaluating the evaluation metrics

Evaluate a few answers produced by the fine-tuned model giving them a score from 1 to 4, where:

1. the answer is completely off
2. the answer is partially correct but not fluent
3. the answer is partially correct but fluent
4. the answer is correct and fluent

Here, we print 5 stories and their respective question-answer pairs, generated by both humans and the QA fine-tuned model. Feel free to evaluate more than 5.

```
# feel free to evaluate more than 5 generated answers,
# it will give you a better sense of the metrics' flaws
N_manual = 5

for i in range(N_manual):
    pprint(f"STORY {i+1}: {dev_set_news_stories[i]}")
    print()
    pprint(f"QUESTION {i+1}: {dev_set_questions[i]}")
    print()
    pprint(f"GENERATED ANSWER {i+1}: {answers_finetuned[i]}")
    print()
    pprint(f"HUMAN ANSWER {i+1}: {dev_set_answers[i]}")
    print('\n-----\n')
```

STORY 1: 'SINDH KALAY', England (CNN) -- The aroma of freshly baking flat British soldiers position themselves for a quick patrol around the villa in the mock Afghan village of Sindh Kalay. Market vendors hawk grapes and smoking water pipes and suspicious-looking men lurk beside battered moto weapons search? Approach the village elders first? In the complex politic the best course of action? Except this is not Afghanistan. It's Norfolk, it is the green landscape and tidy farmhouses of the English countryside British Army's state-of-the art training ground. It cost more than \$20 m serving in Afghanistan will do his or her training here. "I think it's t being in Afghanistan itself," says Col. David Colthup of the 2nd Battali already served one tour of duty in Afghanistan's Helmand province and ar in Helmand province are tasked with mentoring and training Afghan securi stronghold and Afghanistan's center of opium production. "Ultimately, a That's what a soldier trains to do. But today, it's a much, much more co business of being able to interact either through an interpreter or thro police or army. And to understand how the people operate and how we can that's what it's about," he says. The most distinctive features of Sindh up the village compound, creating narrow alleyways difficult for troops to asylum-seekers, many of whom have fled the Taliban. They play the roles o Afghan security forces. Several Afghan women are also on hand, useful fo cultural sensitivities of entering an Afghan home. Watch British troops insurgents are played by Nepalese Ghurkha soldiers authorized to handle to partake in the Pashtun banter among the Afghans. Fazel Beria is also responsible for recruiting and for creating the sights and smells of Sin Afghan in the market in Western clothes. He beams with pride walking dow training the British Army. "Everything with the culture comes up with th you want to win that, you need to know about their culture. You need to way of life." He gives high marks to the soldiers training so far. After to the soldiers about what went wrong and what went right. Sometimes, it have been quite a lot of surprises," Beria says. Like Afghan will sit cr cannot do that," he laughs. "The Afghan will be sitting very comfortable to get used to it." See photos of British troops on patrol in Sindh Kal the army trained on farmhouses and in urban neighborhoods that resembled Sindh Kalay does more than mimic the physical reality of Afghanistan. It ground. Troops are grilled in

QUESTION 1: Where were they being deployed to?

GENERATED ANSWER 1: Helmand province

HUMAN ANSWER 1: Afghanistan's Helmand province

STORY 2: (AOL Autos) -- At the 2009 Detroit Auto Show, Chrysler, Merced electric vehicles of the future will look like. And the future of drivin 0-to-60 mph in around 4 seconds. Those in the know realize that sometime and trucks in the US will feature electric final drive systems. The motor batteries, fuel cells, on-board generators, and perhaps even the sun. Bu inevitability of this reality. Given our current economic times, reality ideas of what electric vehicles (or "EVs" for short) might actually look electric vehicles Chrysler showed in Detroit, Michigan, were shown at ot Each of these vehicles is a running prototype, not some pie-in-the-sky-w group of engineers at Chrysler that develops the company's EVs. To date, powered models, each quite different from the other: a Dodge Circuit EV s Country minivan (front-wheel-drive), a Jeep Wrangler Unlimited (all-whee drive). Chrysler promises to offer at least one of these models in 2010, Circuit EV photos Chrysler approaches electric vehicles with simple plug

Exercise: Provide your scores for the summaries in order in the list below.

```
your_scores = [
    3, # could have mentioned it is in Afganistan
    3, # missed EV
    4, # perfect
    4, # perfect
    4, # wow
]
```

Now let's see what BERTScores they get. Do they match your judgements?

```
bertscores = bertscore.compute(
    predictions=answers_finetuned[:N_manual],
    references=dev_set_answers[:N_manual],
    lang="en"
)
```

```
pprint(f'BertScore: {bertscores}')
```

```
BertScore: {'precision': [0.9158124923706055, 0.8102983236312866, 0.8869
0.962459921836853], 'recall': [0.8827292919158936, 0.8713527917861938, 0
0.9727398753166199], 'f1': [0.8989666104316711, 0.8397172093391418, 0.92
0.9675726294517517], 'hashcode': 'roberta-large_L17_no-idf_version=0.3.1
```

✓ The *Newsreader Chatbot*

Finally all three modules are ready and we can assembled them into our *Newsreader Chatbot*. Enjoy!

```
class NewsReader:

    def __init__(self, database_size=500, num_retrieved_articles=3):
        self.dataset = load_dataset("glnmario/news-qa-summarization")
        self.articles = self.dataset['train']['story'][:database_size]

        self.summarizer = FinetunedSummarizer()
        self.retriever = SentenceEmbeddingRetriever(self.articles, model_na
        self.answerer = FinetunedQA()

        self.num_retrieved_articles = num_retrieved_articles

    def _ask_for_article_number_and_show_article(self, message, selected_
        target_article_idx = input(message)
        while target_article_idx not in map(str, range(1, self.num_retrieve
            target_article_idx = input('\nPlease just enter the article numbe
```

```

        selected_article = selected_articles[int(target_article_idx) - 1]
        pprint(selected_articles[int(target_article_idx) - 1])
        return selected_article

def chat(self):
    query = input('What news topic do you want to look for?\n\n')
    print(f"\nOk! I am searching and summarizing news stories about '{q

    top_indices, top_scores, _ = self.retriever.retrieve(query, top_k=s
    selected_articles = [self.articles[i] for i in top_indices]
    summaries = self.summarizer.summarize(selected_articles)

    print('These are summaries of the three news stories that best matc
    for i, summary in enumerate(summaries, start=1):
        pprint(f'{i}: {summary}')
        print()

    selected_article = self._ask_for_article_number_and_show_article(
        '\nWhich article would you like to read? Please enter the artic
        selected_articles
    )

    question = input('\n\nDo you have any questions?\n\n')
    while question.lower() not in ['no', 'no.', 'nope', "i don't", 'i d
        selected_article = self._ask_for_article_number_and_show_article(
            '\nWould you like to read another article? If so, which one?
            selected_articles
        )
        question = input('\n\nDo you have any questions?\n\n')
        print()
        print(self.answerer.qa([selected_article], [question]))

    print('\nI hope I have been helpful. Talk again soon!')
```

```

bot = NewsReader(database_size=10)
bot.chat()
```

```

Device set to use cuda:0
Initialization: embedding documents...
```

```
100% 10/10 [00:00<00:00, 20.95it/s]
```

```

Device set to use cuda:0
What news topic do you want to look for?
```

```
Autumn
```

```

Ok! I am searching and summarizing news stories about 'Autumn' ...
These are summaries of the three news stories that best match your search
```

```

1: Alaska has the highest gas prices in the nation, with a gallon of reg
Washington make up the top five states with highest prices. Best states t
```

```

2: Sindh Kalay is the British Army's state-of-the-art training ground. T
```

2. Sindh Kalay is the British Army's state of the art training ground. A British soldier will do training here. The village is staffed with Afghan Taliban. Taliban insurgents are played by Nepalese Gurkha soldiers authorized to handle weapons.

3: Three out of four automotive transactions in the U.S. involve previously flood damaged and end up back on the road. The most frequently searched price range is for used Silverado pickup trucks for sale than any other model.

Which article would you like to read? Please enter the article number.

2

'SINDH KALAY', England (CNN) -- The aroma of freshly baking flatbread wafts from the soldiers' position themselves for a quick patrol around the village of Sindh Kalay. Market vendors hawk grapes and melon water pipes and suspicious-looking men lurk beside battered motorcycles. How should you approach the village elders first? In the complex political and cultural landscape, what is the best course of action? Except this is not Afghanistan. It's Norfolk, England, where the green landscape and tidy farmhouses of the English countryside that the British Army's state-of-the-art training ground. It cost more than \$20 million to build, and British soldiers will do their or her training here. "I think it's the closest to Afghanistan itself," says Col. David Colthup of the 2nd Battalion of the Royal Green Jackets, who served one tour of duty in Afghanistan's Helmand province and are training Afghan security forces. Helmand province is also an Afghan security force's center of opium production. "Ultimately, a soldier joins the army to do what a soldier trains to do. But today, it's a much, much more complex environment. You have to be able to interact either through an interpreter or through Afghan security forces to understand how the people operate and how we can interact better with them about," he says. The most distinctive features of Sindh Kalay are the high walls of the compound, creating narrow alleyways difficult for troops to patrol. The village is staffed with many of whom have fled the Taliban. They play the roles of market vendors, security forces, and Afghan women are also on hand, useful for training British soldiers about the sensitivities of entering an Afghan home. Watch British troops training in the village. Taliban insurgents are played by Nepalese Gurkha soldiers authorized to handle weapons. That's the banter among the Afghans. Fazel Beria is also an asylum-seeker who has been recruiting and for creating the sights and smells of Sindh Kalay and is the market in Western clothes. He beams with pride walking down the bazaar at the British Army. "Everything with the culture comes up with the issue of how to win that, you need to know about their culture. You need to respect their life." He gives high marks to the soldiers training so far. After each exercise, he asks the soldiers about what went wrong and what went right. Sometimes, it's the quite a lot of surprises," Beria says. Like Afghan will sit cross-legged on the ground, that," he laughs. "The Afghan will be sitting very comfortable and the British to it." See photos of British troops on patrol in Sindh Kalay -- and find out how they are trained on farmhouses and in urban neighborhoods that resembled Northern Afghanistan. It does more than mimic the physical reality of Afghanistan. It also mirrors the life in Afghanistan.

Do you have any questions?

What is the article about?

Would you like to read another article? If so, which one? Please enter the article number.

would you like to read another article? If so, which one? Please enter t

Where will they be deployed?

Please just enter the article number.

1

(AOL Autos) -- Price of gas got you crying at the pump? Is it eating in Gas in California, the second worst state, hits \$4.59 per gallon for reg word on the streets as the price of a gallon of gasoline soars to record price much higher in some places, drivers and small-business owners are motorists get the best deal, and in which states do drivers pay the most the highest gas prices in the nation, with a gallon of regular gasoline of June 30. Hawaii, Connecticut and Washington make up the top five state not far behind at \$4.292 and \$4.160 respectively. Best states to buy gas of regular, followed by Oklahoma at \$3.866, South Carolina, Mississippi in line. AOL Autos: Cars with 30+ MPG for \$300 or less per month Past pr with this caveat: Back in June 2004, gas hovered at around \$1.74 a gallo Sundstrom, AAA's fuel price analyst, said the primary reasons for rising petroleum products across the globe and a need for investment in refining impossible to know where oil and gas prices will be in six months to a y world economy will continue to grow, we are very clearly at risk of an e downturn which could influence the demand side to the point where oil and oil and gasoline will continue to be more expensive. But given what's ha potential that [the] demands side of the equation could fall off a cliff." efficient used cars Why so expensive in California? Marie Montgomery, spok California, says the primary reason behind the high price of gas in Cali term that basically means California (along with many other states) has uphold environmental or ecological standards. In California's case, this "We can't use gas they made for Iowa. We use a special blend with additi "When you do that, you can only use refineries that make that gas [mix]. supplier, but, when prices are high and there's not another supplier, we pointed out that refineries in Oregon and Washington produce the gas prod that the only other options are shipping in the blend from elsewhere in "People have to figure out how to deal with this on a long-term basis. We we are in summer, and prices have barely come down," she said. "Usually cyclical: Prices go up in spring and spike in May, which isn't happening Less misery in Missouri Michael Right, vice president of public affairs f among the top three in terms of

Do you have any questions?

What is gas price in California?

```
/usr/local/lib/python3.12/dist-packages/transformers/pipelines/question_
warnings.warn(
```

\$4.59 per gallon

Would you like to read another article? If so, which one? Please enter t

1

(AOL Autos) -- Price of gas got you crying at the pump? Is it eating in Gas in California the second worst state hits \$4.59 per gallon for reg

gas in California, the second worst state, hits \$4.99 per gallon for regular word on the streets as the price of a gallon of gasoline soars to record price much higher in some places, drivers and small-business owners are motorists get the best deal, and in which states do drivers pay the most the highest gas prices in the nation, with a gallon of regular gasoline of June 30. Hawaii, Connecticut and Washington make up the top five state not far behind at \$4.292 and \$4.160 respectively. Best states to buy gas of regular, followed by Oklahoma at \$3.866, South Carolina, Mississippi in line. AOL Autos: Cars with 30+ MPG for \$300 or less per month Past price with this caveat: Back in June 2004, gas hovered at around \$1.74 a gallon Sundstrom, AAA's fuel price analyst, said the primary reasons for rising petroleum products across the globe and a need for investment in refining impossible to know where oil and gas prices will be in six months to a year world economy will continue to grow, we are very clearly at risk of an economic downturn which could influence the demand side to the point where oil and oil and gasoline will continue to be more expensive. But given what's happening potential that [the] demand side of the equation could fall off a cliff." efficient used cars Why so expensive in California? Marie Montgomery, spokeswoman California, says the primary reason behind the high price of gas in California term that basically means California (along with many other states) has uphold environmental or ecological standards. In California's case, this "We can't use gas they made for Iowa. We use a special blend with additives "When you do that, you can only use refineries that make that gas [mix]. supplier, but, when prices are high and there's not another supplier, we pointed out that refineries in Oregon and Washington produce the gas product that the only other options are shipping in the blend from elsewhere in "People have to figure out how to deal with this on a long-term basis. We we are in summer, and prices have barely come down," she said. "Usually cyclical: Prices go up in spring and spike in May, which isn't happening Less misery in Missouri Michael Right, vice president of public affairs for among the top three in terms of

Do you have any questions?

What will happen to demand for gas?

fall off a cliff

Would you like to read another article? If so, which one? Please enter the

5

Please just enter the article number.

3

(AOL Autos) -- Buying a used car may seem like a simple topic. Used cars and SUVs after gas prices skyrocketed. But when you begin slicing and dicing approximately 3.3 million vehicles that are for sale at any given time, they begin to emerge. For Krista Glotzbach, director of marketing at Vast.com, a trove of data (which provides used vehicle search data for AOL Autos), the variety shouldn't worry because, with the help of search filters, vehicle shopping budgetary considerations (what price range do you want) easily come to the fore. Because Vast.com has a variety of sources for its data, including Web 2.0

Because vast.com has a variety of sources for its data, including web (c
vehicle information), the data is, by definition, revealing in its own ri
CARFAX, which makes its vehicle history reports available to the public,
Autos: Used luxury cars As Larry Gamache, director of communications, po
automotive industry. With that volume comes its share of fraud, notably
And even if a vehicle doesn't look bad, what you can't see can hurt. "Th
they're rotting from the inside out," Gamache said. It's also a potentia
submerged do not function properly," Gamache noted. "They either don't d
used cars have skeletons in their closets. Actually, most of the vehicle
popular crossovers Here are a few water cooler tidbits that might make y
transactions in the U.S. involve previously owned vehicles. 2. The avera
lifetime. 3. Industry experts believe one of every 25 air bags that have
They're also the most stolen item from a vehicle (having surpassed radio
half the cars that are flood damaged end up back on the road. As far as H
that about 200,000 vehicles were dried out and resold. 5. The most frequ
under \$5,000. 6. Consumers lose as much as \$4 billion a year due to odom
15,000 miles. AOL Autos: Best hybrid cars 7. There are more used Silvera
(nearly 120,000 throughout the U.S.) 8. Texas has more used vehicles tha
alone has more than 32,000 on the market at any given time. 9. Chevrolet
U.S. (with about 450,000 available), followed by Ford (425,000). There a
any given time. 10. Silver is the most available color in the used vehic
followed by black (429,000), white (409,000), and red (321,000). Blue (3
color you're least likely to find? Pink, with just 249 used vehicles list
when it comes to a used vehicle. Nearly half those listed at any one tim
12. Automatic transmissions outnumber manual 10 to 1 in the market for u
most popular on

Do you have any questions?

What is the article about

Buying a used car

Would you like to read another article? If so, which one? Please enter t

4

Please just enter the article number.

3

(AOL Autos) -- Buying a used car may seem like a simple topic. Used car
and SUVs after gas prices skyrocketed. But when you begin slicing and di
approximately 3.3 million vehicles that are for sale at any given time,
begin to emerge. For Krista Glotzbach, director of marketing at Vast.com
of data (which provides used vehicle search data for AOL Autos), the var
shouldn't worry because, with the help of search filters, vehicle fashion
budgetary considerations (what price range do you want) easily come to t
Because Vast.com has a variety of sources for its data, including Web "c
vehicle information), the data is, by definition, revealing in its own ri
CARFAX, which makes its vehicle history reports available to the public,
Autos: Used luxury cars As Larry Gamache, director of communications, po
automotive industry. With that volume comes its share of fraud, notably
And even if a vehicle doesn't look bad, what you can't see can hurt. "Th

And even if a vehicle doesn't look bad, what you can't see can hurt. "If they're rotting from the inside out," Gamache said. It's also a potential submerged do not function properly," Gamache noted. "They either don't used cars have skeletons in their closets. Actually, most of the vehicle popular crossovers Here are a few water cooler tidbits that might make y transactions in the U.S. involve previously owned vehicles. 2. The average lifetime. 3. Industry experts believe one of every 25 air bags that have They're also the most stolen item from a vehicle (having surpassed radio half the cars that are flood damaged end up back on the road. As far as H that about 200,000 vehicles were dried out and resold. 5. The most frequent under \$5,000. 6. Consumers lose as much as \$4 billion a year due to odometer 15,000 miles. AOL Autos: Best hybrid cars 7. There are more used Silverado (nearly 120,000 throughout the U.S.) 8. Texas has more used vehicles than alone has more than 32,000 on the market at any given time. 9. Chevrolet U.S. (with about 450,000 available), followed by Ford (425,000). There are any given time. 10. Silver is the most available color in the used vehicle followed by black (429,000), white (409,000), and red (321,000). Blue (3 color you're least likely to find? Pink, with just 249 used vehicles listed when it comes to a used vehicle. Nearly half those listed at any one time 12. Automatic transmissions outnumber manual 10 to 1 in the market for the most popular on

Do you have any questions?

How to buy a used car

skeletons in their closets

Would you like to read another article? If so, which one? Please enter the

3

(AOL Autos) -- Buying a used car may seem like a simple topic. Used cars and SUVs after gas prices skyrocketed. But when you begin slicing and dicing approximately 3.3 million vehicles that are for sale at any given time, you begin to emerge. For Krista Glotzbach, director of marketing at Vast.com of data (which provides used vehicle search data for AOL Autos), the variety shouldn't worry because, with the help of search filters, vehicle fashion budgetary considerations (what price range do you want) easily come to the fore. Because Vast.com has a variety of sources for its data, including Web "c vehicle information), the data is, by definition, revealing in its own right. CARFAX, which makes its vehicle history reports available to the public, AOL Autos: Used luxury cars As Larry Gamache, director of communications, for the automotive industry. With that volume comes its share of fraud, notably And even if a vehicle doesn't look bad, what you can't see can hurt. "If they're rotting from the inside out," Gamache said. It's also a potential submerged do not function properly," Gamache noted. "They either don't used cars have skeletons in their closets. Actually, most of the vehicle popular crossovers Here are a few water cooler tidbits that might make y transactions in the U.S. involve previously owned vehicles. 2. The average lifetime. 3. Industry experts believe one of every 25 air bags that have They're also the most stolen item from a vehicle (having surpassed radio half the cars that are flood damaged end up back on the road. As far as H that about 200,000 vehicles were dried out and resold. 5. The most frequent

that about 200,000 vehicles were dried out and resold. 5. The most frequ under \$5,000. 6. Consumers lose as much as \$4 billion a year due to odom 15,000 miles. AOL Autos: Best hybrid cars 7. There are more used Silvera (nearly 120,000 throughout the U.S.) 8. Texas has more used vehicles tha alone has more than 32,000 on the market at any given time. 9. Chevrolet U.S. (with about 450,000 available), followed by Ford (425,000). There a any given time. 10. Silver is the most available color in the used vehic followed by black (429,000), white (409,000), and red (321,000). Blue (3 color you're least likely to find? Pink, with just 249 used vehicles list when it comes to a used vehicle. Nearly half those listed at any one tim 12. Automatic transmissions outnumber manual 10 to 1 in the market for u most popular on

Do you have any questions?

Are there a lot of Toyotas on the market

There are "only" about 280,000

Would you like to read another article? If so, which one? Please enter t

3

(AOL Autos) -- Buying a used car may seem like a simple topic. Used car and SUVs after gas prices skyrocketed. But when you begin slicing and di approximately 3.3 million vehicles that are for sale at any given time, begin to emerge. For Krista Glotzbach, director of marketing at Vast.com of data (which provides used vehicle search data for AOL Autos), the var shouldn't worry because, with the help of search filters, vehicle fashion budgetary considerations (what price range do you want) easily come to t Because Vast.com has a variety of sources for its data, including Web "c vehicle information), the data is, by definition, revealing in its own ri CARFAX, which makes its vehicle history reports available to the public, Autos: Used luxury cars As Larry Gamache, director of communications, po automotive industry. With that volume comes its share of fraud, notably And even if a vehicle doesn't look bad, what you can't see can hurt. "Th they're rotting from the inside out," Gamache said. It's also a potentia submerged do not function properly," Gamache noted. "They either don't d used cars have skeletons in their closets. Actually, most of the vehicle popular crossovers Here are a few water cooler tidbits that might make y transactions in the U.S. involve previously owned vehicles. 2. The avera lifetime. 3. Industry experts believe one of every 25 air bags that have They're also the most stolen item from a vehicle (having surpassed radio half the cars that are flood damaged end up back on the road. As far as H that about 200,000 vehicles were dried out and resold. 5. The most frequ under \$5,000. 6. Consumers lose as much as \$4 billion a year due to odom 15,000 miles. AOL Autos: Best hybrid cars 7. There are more used Silvera (nearly 120,000 throughout the U.S.) 8. Texas has more used vehicles tha alone has more than 32,000 on the market at any given time. 9. Chevrolet U.S. (with about 450,000 available), followed by Ford (425,000). There a any given time. 10. Silver is the most available color in the used vehic followed by black (429,000), white (409,000), and red (321,000). Blue (3 color you're least likely to find? Pink, with just 249 used vehicles list when it comes to a used vehicle. Nearly half those listed at any one tim

when it comes to a used vehicle. Nearly half chose listed as any one car.
 12. Automatic transmissions outnumber manual 10 to 1 in the market for u
 most popular on

Do you have any questions?

What are best hybrid cars

AOL Autos: Best hybrid cars 7

Would you like to read another article? If so, which one? Please enter t

No

Please just enter the article number.

no

Please just enter the article number.

None

Please just enter the article number.

1

(AOL Autos) -- Price of gas got you crying at the pump? Is it eating in
 Gas in California, the second worst state, hits \$4.59 per gallon for reg
 word on the streets as the price of a gallon of gasoline soars to record
 price much higher in some places, drivers and small-business owners are
 motorists get the best deal, and in which states do drivers pay the most
 the highest gas prices in the nation, with a gallon of regular gasoline
 of June 30. Hawaii, Connecticut and Washington make up the top five state
 not far behind at \$4.292 and \$4.160 respectively. Best states to buy gas
 of regular, followed by Oklahoma at \$3.866, South Carolina, Mississippi
 in line. AOL Autos: Cars with 30+ MPG for \$300 or less per month Past pr
 with this caveat: Back in June 2004, gas hovered at around \$1.74 a gallo
 Sundstrom, AAA's fuel price analyst, said the primary reasons for rising
 petroleum products across the globe and a need for investment in refining
 impossible to know where oil and gas prices will be in six months to a y
 world economy will continue to grow, we are very clearly at risk of an e
 downturn which could influence the demand side to the point where oil and
 oil and gasoline will continue to be more expensive. But given what's ha
 potential that [the] demands side of the equation could fall off a cliff."
 efficient used cars Why so expensive in California? Marie Montgomery, spok
 California, says the primary reason behind the high price of gas in Cali
 term that basically means California (along with many other states) has
 uphold environmental or ecological standards. In California's case, this
 "We can't use gas they made for Iowa. We use a special blend with additi
 "When you do that, you can only use refineries that make that gas [mix].
 supplier, but, when prices are high and there's not another supplier, we
 pointed out that refineries in Oregon and Washington produce the gas prod
 that the only other options are shipping in the blend from elsewhere in
 "People have to figure out how to deal with this on a long-term basis. We

People have to figure out how to deal with this on a long-term basis. We are in summer, and prices have barely come down," she said. "Usually cyclical: Prices go up in spring and spike in May, which isn't happening. Less misery in Missouri. Michael Right, vice president of public affairs for among the top three in terms of

Do you have any questions?

No

where oil and gas prices will be in six months to a year